

From Zero to Propagators

Orders, Lattices, and Propagator Networks in Lean 4

*“The propagator model is to functional programming
what constraint logic programming is to logic programming:
a radical generalisation that turns programs into networks
of co-operating, partial knowledge sources.”*

— Inspired by Radul & Sussman (2009)

A Self-Contained Journey Through Order Theory
to Verified Constraint Propagation

Lean 4 code requires no external libraries beyond the standard prelude.

Edition 1.0

Preface

This book has a single grand ambition: to take you from the very first principles of ordered structures all the way to building a working *propagator network* — a powerful computational model that arises naturally from the mathematics of lattices — and to do it with complete rigour in Lean 4.

Who this book is for. We assume you are comfortable reading mathematics at the level of a second-year undergraduate: sets, functions, logical quantifiers, and basic proof techniques are fair game. No prior knowledge of abstract algebra or Lean is required, though some familiarity with functional programming will help when reading the longer code sections.

The Lean 4 philosophy here. Every definition and theorem in this book has a corresponding Lean 4 encoding. We deliberately avoid Mathlib to keep the dependency footprint at zero and to force you to understand every axiom rather than importing it from a library. Where a Mathlib equivalent exists we point it out in a margin note.

How to read this book. Part I builds the algebraic foundations: relations, partial orders, and lattices. Part II shows how these structures give rise to a beautiful computational model — the propagator network. Part III contains two capstone projects that demonstrate the full power of the model: a Sudoku solver driven entirely by constraint propagation, and a type-inference engine for a small lambda calculus. By the time you reach the end of Part III the connection between the mathematics and the code should feel inevitable.

A note on Lean Unicode. All code listings use the Unicode characters that Lean 4 accepts natively: $\forall$ $\exists$ $\rightarrow$ $\leftarrow$ $\leftrightarrow$ $\leq$ $\geq$ $\sqcap$ $\sqcup$ $\perp$ $\top$ $\in$ $\notin$ α β γ $\times$ and so on. When reading the code, treat these as single characters exactly as Lean treats them. In Lean 4 you enter them with backslash sequences (e.g. `\le` for $\leq$); your editor should substitute them live.

Contents

Preface	iii
I The Algebra of Order	1
1 Relations and Partial Orders	3
1.1 Binary Relations	3
1.2 Properties of Relations	4
1.3 Partial Orders	4
1.3.1 The Natural Numbers	5
1.3.2 Divisibility	6
1.3.3 Sets	10
1.4 Hasse Diagrams	11
1.5 Computation Interlude: Running the Divisibility Order	12
1.6 Exercises	13
2 Special Elements and Monotone Maps	15
2.1 Extremal Elements	15
2.2 Upper and Lower Bounds	17
2.3 Monotone Maps	18
2.4 Exercises	19
2.5 Computation Interlude: Running Monotone Maps	19
3 Lattices	23
3.1 Meet and Join	23
3.1.1 Derived Properties	24
3.2 The Duality Principle	25
3.3 Important Lattice Examples	27
3.3.1 The Two-Element Lattice 2	27
3.3.2 The Powerset Lattice	28
3.3.3 The Divisibility Lattice on Divisors of n	29
3.4 Distributive Lattices	29
3.5 Exercises	30
4 Complete Lattices and the Fixed-Point Theorem	33
4.1 Complete Lattices	33
4.2 Why Fixed Points Matter	34

4.3	The Knaster–Tarski Fixed-Point Theorem	35
4.4	Chains and Convergence	36
4.5	Exercises	38
II	Propagator Networks	39
5	The Propagator Model	41
5.1	Cells: Accumulating Partial Information	41
5.2	Why Information Can Only Increase	42
5.3	Propagators as Monotone Functions	42
5.4	Networks, Scheduling, and Quiescence	43
5.5	The Fundamental Role of the Lattice	44
6	Building a Propagator Network in Lean 4	47
6.1	Interlude: Imperative Lean in Half a Page	47
6.2	Mutable Cells	48
6.3	The Network and Scheduler	49
6.4	Building Propagators: Numeric Example	49
6.5	Monotonicity Proofs for <code>addProp</code> and <code>subProp</code>	51
6.6	A Complete Running Example	52
6.7	Why the Lattice Is Load-Bearing Here	53
6.8	Exercises	54
7	The Interval Lattice	55
7.1	Intervals as Partial Information	55
7.2	Bounds: Option Int as the Extended Integers	56
7.3	The Interval Lattice in Lean	59
7.4	Interval Arithmetic Propagators	63
7.5	A Worked Example: Bound Propagation	64
7.6	Exercises	66
III	Capstone Projects	69
8	Capstone I: A Sudoku Solver	71
8.1	What is Sudoku?	71
8.2	The Domain Lattice	72
8.3	Representing the Grid	75
8.4	Constraint Propagators	75
8.5	Assembling and Running the Solver	77
8.6	Why This Works: The Lattice Perspective	79
9	Capstone II: Type Inference via Propagation	81
9.1	A Simple Expression Language	81

9.2	The Type Lattice	81
9.3	Type Constraint Generation	84
9.4	Running the Type Inferencer	87
9.5	What the Lattice Buys Us Here	88
9.6	Exercises	88
A	Lean 4 Quick Reference and Tactic Primer	91
A.1	Basic Types and Syntax	91
A.2	Typeclasses	91
A.3	Tactic Primer	92
A.4	Programming Constructs	98
A.5	Tactics at a Glance	99
A.6	Unicode Input	100
	Selected Solutions	103
	Further Reading	109

Part I

The Algebra of Order

Chapter 1

Relations and Partial Orders

“The concept of an ordering is one of the most basic in all of mathematics, and yet we rarely pause to examine what it actually requires.”

Before we can study lattices or build propagator networks, we need a precise vocabulary for talking about *order*: what it means for one thing to be “at most” another, how such a concept can be captured in a type, and which algebraic laws make it useful.

1.1 Binary Relations

Definition — Binary Relation

A *binary relation* on a set A is a subset $R \subseteq A \times A$. We write $a R b$ (or $R(a, b)$) as shorthand for $(a, b) \in R$.

Lean 4 encodes a relation as a function returning **Prop**:

```
1 -- A binary relation on type  $\alpha$ 
2 def Rel ( $\alpha$  : Type) : Type :=  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ 
```

Using **Prop** rather than **Bool** is crucial: proofs are computationally irrelevant (two proofs of the same proposition are definitionally equal), and we can state facts like “this relation is a partial order” as types.

Example — Common relations

- $\leq$ on $\mathbb{N}$: $R = \{(m, n) \mid m \leq n\}$.
- Divisibility on $\mathbb{N}$: $m \mid n \iff \exists k, n = m \cdot k$.
- Subset inclusion on $\mathcal{P}(S)$: $A \subseteq B \iff \forall x, x \in A \rightarrow x \in B$.
- The equality relation $=$ on any set.
- The *discrete* relation: $a R b \iff a = b$.
- The *trivial* relation: $a R b$ for all a, b (everything relates to everything).

1.2 Properties of Relations

Four properties will occupy us throughout the book.

Definition — Reflexivity, Symmetry, Antisymmetry, Transitivity

Let R be a binary relation on A .

$$\begin{aligned} \text{Reflexive} &: \iff \forall a \in A, a R a \\ \text{Symmetric} &: \iff \forall a, b \in A, a R b \rightarrow b R a \\ \text{Antisymmetric} &: \iff \forall a, b \in A, a R b \rightarrow b R a \rightarrow a = b \\ \text{Transitive} &: \iff \forall a, b, c \in A, a R b \rightarrow b R c \rightarrow a R c \end{aligned}$$

In Lean 4:

```

1 namespace OrderBook
2
3 def Reflexive {α : Type} (r : Rel α) : Prop :=
4   ∀ (a : α), r a a
5
6 def Symmetric {α : Type} (r : Rel α) : Prop :=
7   ∀ (a b : α), r a b → r b a
8
9 def Antisymm {α : Type} (r : Rel α) : Prop :=
10  ∀ (a b : α), r a b → r b a → a = b
11
12 def Transitive {α : Type} (r : Rel α) : Prop :=
13  ∀ (a b c : α), r a b → r b c → r a c
14
15 end OrderBook

```

Symmetry vs Antisymmetry. These are not negations of each other! A relation can be both symmetric and antisymmetric (equality), neither (“ a is a parent of b ”), or only one. Antisymmetry says: if you can go *both* ways, the elements must be equal.

1.3 Partial Orders

Definition — Partial Order

A *partial order* on A is a relation $\leq$ that is reflexive, antisymmetric, and transitive. The pair $(A, \leq)$ is called a *partially ordered set* (or *poset*).

The word *partial* refers to the fact that not every two elements need be comparable:

there may exist a, b with neither $a \leq b$ nor $b \leq a$.

In Lean 4 we capture this as a typeclass:

```

1 namespace OrderBook
2
3 -- Our own partial order typeclass (mirrors core Lean / Mathlib).
4 class PartialOrder (α : Type) where
5   le      : α → α → Prop
6   le_refl : ∀ (a : α), le a a
7   le_antisymm : ∀ (a b : α), le a b → le b a → a = b
8   le_trans  : ∀ (a b c : α), le a b → le b c → le a c
9
10 -- Attach the ≤ notation to any PartialOrder instance.
11 instance [PartialOrder α] : LE α := ⟨PartialOrder.le⟩
12
13 -- A convenience alias for strict less-than
14 def lt [PartialOrder α] (a b : α) : Prop :=
15   PartialOrder.le a b ∧ ¬ (a = b)
16
17 instance [PartialOrder α] : LT α := ⟨lt⟩
18
19 end OrderBook

```

A naming convention. Definitions like `PartialOrder` live in our `OrderBook` namespace so they cannot collide with core Lean's. Listings that sit outside an explicit `namespace OrderBook ... end` block assume `open OrderBook` is in effect (occasionally written explicitly as `open OrderBook in` before a single declaration), which lets us write `PartialOrder.le\_antisymm` instead of the fully qualified `OrderBook.PartialOrder.le\_antisymm`.

1.3.1 The Natural Numbers

The natural numbers with the usual $\leq$ form our first instance. In Lean 4's core library, `Nat.le` is defined inductively:

```

1 -- Recap: how ≤ is defined in core Lean 4 (no Mathlib needed)
2 -- inductive Nat.le : Nat → Nat → Prop
3 --   | refl : Nat.le n n
4 --   | step : Nat.le n m → Nat.le n (m + 1)
5
6 instance : OrderBook.PartialOrder Nat where
7   le      := (· ≤ ·)      -- uses Nat.le which is in core
8   le_refl := Nat.le_refl
9   le_antisymm := fun a b h1 h2 => Nat.le_antisymm h1 h2

```

```
10 le_trans := fun a b c h1 h2 => Nat.le_trans h1 h2
```

Lean 4's *core* library already provides `Nat.le_refl`, `Nat.le_antisymm`, and `Nat.le_trans`. We reuse these rather than reprove them. In later chapters we will prove non-trivial instances by hand.

1.3.2 Divisibility

Divisibility is a partial order on $\mathbb{N}$: $n \leq m$ means “ n divides m ”, i.e. $\exists k, m = n \cdot k$. This is a genuinely different order from $\leq$: for instance $3 \leq 12$ (since $12 = 3 \cdot 4$) but $4 \not\leq 6$ (no natural number k satisfies $6 = 4k$).

Core Lean already attaches an `LE Nat` instance for the usual order, and adding a second one for divisibility would make every $\leq$ on `Nat` ambiguous. So we introduce a newtype wrapper and give *it* the divisibility order.

Why `structure` and not `def DvdNat := Nat`? Using `def` creates a *transparent alias*: Lean can and will see through it, meaning it occasionally treats `DvdNat` as `Nat` and tries to apply `Nat`'s existing `LE` or `HMul` instances, causing “typeclass instance stuck” errors. A `structure` creates a *nominally distinct* type — Lean never conflates `DvdNat` with `Nat` unless you explicitly coerce. This is the reliable choice whenever you want a newtype for instance purposes.

```
1 -- A nominally distinct wrapper type for Nat under divisibility
2 structure DvdNat where
3   val : Nat
4
5 -- Coercion from DvdNat to Nat (one direction is enough for our proofs)
6 instance : Coe DvdNat Nat := ⟨DvdNat.val⟩
7
8 -- Numeric literals: (5 : DvdNat) desugars to DvdNat.mk 5
9 instance {n : Nat} : OfNat DvdNat n := ⟨{n}⟩
10
11 -- The divisibility relation, stated entirely in Nat to avoid ambiguity
12 -- in the multiplication on the right-hand side.
13 def Dvd' (n m : DvdNat) : Prop := ∃ k : Nat, m.val = n.val * k
14
15 instance : LE DvdNat := ⟨Dvd'⟩
```

By using `.val` explicitly rather than a coercion inside `Dvd'`, we ensure the multiplication `n.val * k` is unambiguously `Nat` multiplication. Coercions are useful for moving single values across a boundary, but inside an expression with multiple interacting terms they give Lean too many degrees of freedom and can produce stuck typeclass searches.

Building up the proof: the lemmas we need

The reflexivity and transitivity proofs are short. Antisymmetry — “if $a \mid b$ and $b \mid a$ then $a = b$ ” — is where the real mathematical content lives. Rather than writing it as a single inscrutable block, we build it from three small lemmas, each of which is worth understanding on its own.

First contact with tactics. The proofs below are our first *tactic proofs*: everything after `by` is a sequence of commands (`obtain`, `simp`, `exact`, ...) that transform the goal step by step. Every tactic and programming construct used anywhere in this book has an entry — with a small worked example, and where illuminating, the term it desugars to — in the primer of Appendix A. Keep it bookmarked; we will not re-explain a tactic after its first appearance.

Lemma 1: if $a = 0$ and $a \mid b$, then $b = 0$.

If $a = 0$, then $b = 0 \cdot k = 0$ for any k . This is the zero case of antisymmetry: the only thing that 0 divides is 0 itself.

```

1  -- If 0 | m, then m.val = 0.
2  -- Proof: the witness k gives m.val = 0 * k.
3  -- We then reduce 0 * k to 0 using Nat.zero_mul.
4  theorem dvd_zero_left (m : DvdNat) (h : Dvd' {0} m) : m.val = 0 := by
5    obtain ⟨k, hk⟩ := h
6    -- hk : m.val = DvdNat.mk(0).val * k
7    -- DvdNat.mk(0).val reduces to 0 by definition, so simp can see it
8    simp at hk
9    -- hk is now: m.val = 0
10   exact hk

```

Why `simp` instead of `rw [Nat.zero_mul]`? With a `structure`, the literal `0 : DvdNat` is `OfNat.ofNat 0 = DvdNat.mk 0`, so its `.val` is `DvdNat.mk(0).val`, not the bare `0` that `Nat.zero_mul` expects as its left-hand side pattern. `simp` performs the definitional unfolding first and then applies `Nat.zero_mul`, where a bare `rw` cannot. If you want to avoid `simp` you can write `show m.val = 0` after an explicit `have : DvdNat.mk(0).val = 0 := rfl`.

Lemma 2: if $k \cdot j = 1$ in $\mathbb{N}$, then $k = 1$.

This is the key algebraic fact that drives the nonzero case. In $\mathbb{Z}$ the units are ± 1 ; in $\mathbb{N}$ the only unit is 1. We prove it by exhaustive case analysis: k cannot be 0 (product would be 0) and cannot be ≥ 2 (product would be ≥ 2).

```

1  -- In ℕ, k * j = 1 forces k = 1 (and by symmetry, j = 1).
2  -- Proved by explicit case analysis on k; omega only closes the final
3  -- numeric contradiction. NB: the calc chain is written in the ≤

```

```

4 -- direction – core Lean ships a Trans instance for ≤ but not for ≥.
5 theorem mul_eq_one_left {k j : Nat} (h : k * j = 1) : k = 1 := by
6   match k with
7   | 0 =>
8     -- 0 * j = 0 ≠ 1, so this case is impossible.
9     -- simp knows 0 * j = 0, which contradicts h : 0 = 1.
10    simp [Nat.zero_mul] at h
11  | 1 =>
12    -- k is already 1; no further argument needed.
13    rfl
14  | k + 2 =>
15    -- (k+2) * j ≥ 2, contradicting h : (k+2) * j = 1.
16    exfalso
17    -- First establish j ≥ 1 (if j = 0, the product is 0, not 1).
18    have hj : j ≥ 1 := by
19      cases j with
20      | zero    => simp [Nat.mul_zero] at h
21      | succ j => exact Nat.succ_le_succ (Nat.zero_le j)
22    -- Now show the product is at least 2.
23    have hbig : 2 ≤ (k + 2) * j :=
24      calc 2 ≤ k + 2          := Nat.le_add_left 2 k
25            _ = (k + 2) * 1    := (Nat.mul_one (k + 2)).symm
26            _ ≤ (k + 2) * j    := Nat.mul_le_mul_left (k + 2) hj
27    -- h says the product is 1, but hbig says it is ≥ 2. Contradiction.
28    omega
29
30 -- The right-factor version follows by commutativity.
31 theorem mul_eq_one_right {k j : Nat} (h : k * j = 1) : j = 1 :=
32   mul_eq_one_left (by rw [Nat.mul_comm]; exact h)

```

Lemma 3: if $a > 0$, $a \mid b$, and $b \mid a$, then $a = b$.

Given witnesses k and j with $b = a \cdot k$ and $a = b \cdot j$, we substitute to get $a = a \cdot (k \cdot j)$, cancel a (valid because $a > 0$), apply Lemma 2, and conclude $k = 1$, hence $b = a$.

```

1 -- All arithmetic here lives in Nat – no DvdNat involved.
2 theorem dvd_antisymm_pos
3   {a b : Nat} (ha : 0 < a)
4   (k : Nat) (hk : b = a * k)
5   (j : Nat) (hj : a = b * j) : a = b := by
6   -- Step 1: derive k * j = 1.
7   -- Substitute hk into hj to get a = a * (k * j),
8   -- then cancel a (valid since a > 0).
9   have hkj : k * j = 1 := by
10    have hstep : a = a * (k * j) :=
11      calc a = b * j          := hj
12            _ = (a * k) * j := by rw [hk]

```



```

13     _ = a * (k * j) := by rw [Nat.mul_assoc]
14     exact Nat.eq_of_mul_eq_mul_left ha (by rw [Nat.mul_one]; exact hstep.symm)
15     -- Step 2: j = 1 by Lemma 2 (right factor version).
16     have hj1 : j = 1 := mul_eq_one_right hkj
17     -- Step 3: substitute j = 1 into hj : a = b * j to get a = b * 1 = b.
18     have h2 : a = b * 1 := by rw [hj1] at hj; exact hj
19     rw [Nat.mul_one] at h2
20     exact h2

```

Why `Nat.eq_of_mul_eq_mul_left`? This says: if $a > 0$ and $a \cdot x = a \cdot y$, then $x = y$. It is multiplication cancellation for $\mathbb{N}$, available in core Lean. The positivity hypothesis is essential: $0 \cdot x = 0 \cdot y$ for all x, y , so cancellation is unsound at zero.

Assembling the instance

With the lemmas in hand, each field is a short delegation.

```

1 open OrderBook in
2 instance : PartialOrder DvdNat where
3
4   -- The order itself: divisibility.
5   le := Dvd'
6
7   -- Reflexivity: n | n via witness k = 1, since n.val = n.val * 1.
8   le_refl := fun n =>
9     (1, (Nat.mul_one n.val).symm)
10
11  -- Antisymmetry: expose the divisibility witnesses, then case-split
12  -- on whether n.val is zero or positive, delegating to Lemma 1 or
13  -- Lemma 3 respectively.
14  le_antisymm := fun n m hnm hmn => by
15    obtain ⟨k, hk⟩ := hnm
16    obtain ⟨j, hj⟩ := hmn
17    -- Reduce n = m (DvdNat equality) to a = b on the .val fields:
18    -- cases exposes the fields, congr 1 splits the constructor.
19    cases n; cases m
20    congr 1
21    rename_i a b
22    cases Nat.eq_zero_or_pos a with
23    | inl ha =>
24      -- a = 0: b = 0 * k = 0 by Lemma 1, and a = 0, so a = b.
25      subst ha
26      exact (dvd_zero_left ⟨b⟩ ⟨k, hk⟩).symm
27    | inr ha =>
28      -- a > 0: use Lemma 3 directly.
29      exact dvd_antisymm_pos ha k hk j hj

```

```

30
31 -- Transitivity: compose witnesses.  b.val = a.val * k and
32 -- c.val = b.val * j, so c.val = a.val * (k * j).
33 le_trans := fun a b c ⟨k, hk⟩ ⟨j, hj⟩ =>
34   ⟨k * j, by rw [hj, hk, Nat.mul_assoc]⟩

```

Three details worth noticing. (1) Our `PartialOrder` class bundles the order itself as the `le` field, so the instance must supply it even though we also declared an `LE DvdNat` instance above — that one only provides the $\leq$ notation. (2) We destructure the hypotheses with `obtain` inside the tactic block rather than in the lambda binders: pattern-matching binders like `fun n m ⟨k, hk⟩ ⟨j, hj⟩ =>` would leave the witnesses inaccessible to `rename\_i` after `cases n; cases m`. (3) `dvd\_antisymm\_pos` already concludes $a = b$, exactly the goal, so no `.symm` is needed in the positive case.

Reflection

Notice that the structure of the proof exactly mirrors the structure of the mathematics:

- Reflexivity is trivial because the witness $k = 1$ is explicit.
- Transitivity is trivial because multiplying witnesses gives a new witness: $b = a \cdot k$ and $c = b \cdot j$ compose to $c = a \cdot (kj)$.
- Antisymmetry requires real work because it cannot be witnessed directly — it requires us to reason about what witnesses can possibly exist and conclude they must all be 1.

The lemma `mul\_eq\_one\_left` is the mathematical core. Everything else is bookkeeping. When reading an unfamiliar proof, it is always worth asking: where is the actual mathematical content? In this case it is entirely in that one lemma.

1.3.3 Sets

Sets (represented as predicates) form a canonical partial order under inclusion.

```

1 -- We represent sets as indicator functions
2 def Set (α : Type) : Type := α → Prop
3
4 -- Subset inclusion
5 def Set.subset {α : Type} (s t : Set α) : Prop :=
6   ∀ (a : α), s a → t a
7
8 instance {α : Type} : OrderBook.PartialOrder (Set α) where
9   le      := Set.subset
10  le_refl := fun s a ha => ha
11  le_antisymm := fun s t h1 h2 =>
12    funext (fun a => propext ⟨h1 a, h2 a⟩)

```

```
13 le_trans := fun s t u h1 h2 a ha => h2 a (h1 a ha)
```

This is our first non-trivial Lean proof. Let us read it carefully:

- `le_refl`: given $a \in s$, produce $a \in s$ — trivial.
- `le_antisymm`: $s \subseteq t$ and $t \subseteq s$ imply $s = t$. We use `funext` (function extensionality) and `propext` (propositional extensionality) to conclude $s a \leftrightarrow t a$ for all a , hence $s = t$.
- `le_trans`: chain the inclusions.

1.4 Hasse Diagrams

Posets are best understood visually. A *Hasse diagram* draws elements as nodes and covers as upward edges: we say a is *covered by* b (written $a < b$) if $a < b$ and there is no c with $a < c < b$.

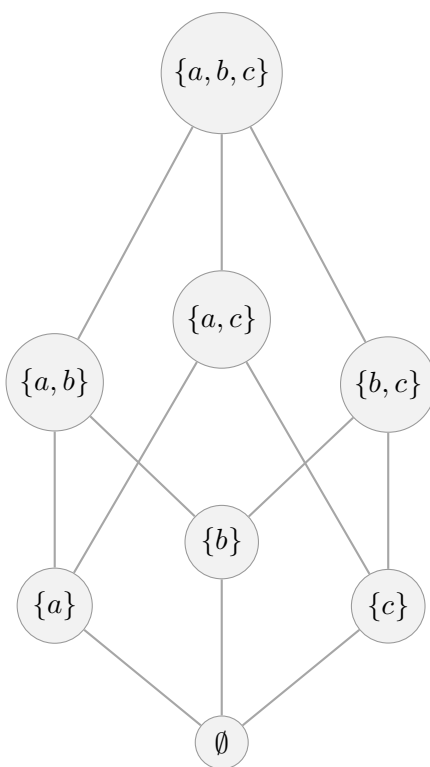
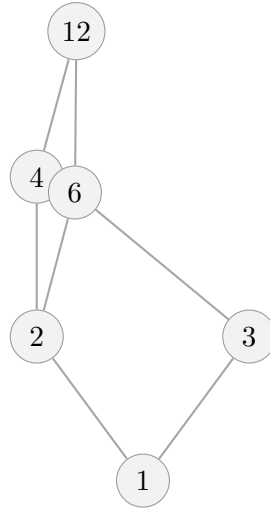


Figure 1.1: The powerset lattice $\mathcal{P}(\{a, b, c\})$ ordered by $\subseteq$.

Figure 1.2: The divisibility poset on $\{1, 2, 3, 4, 6, 12\}$.

1.5 Computation Interlude: Running the Divisibility Order

So far every definition has lived in **Prop**. This section bridges that gap with a fully computable divisibility order on $\{1, \dots, 12\}$.

```

1 def D12 : List Nat := [1, 2, 3, 4, 6, 12]
2 def dvd12 (a b : Nat) : Bool := b % a == 0
3
4 -- Supremum: smallest upper bound
5 def upperBounds (xs : List Nat) : List Nat :=
6   D12.filter (fun u => xs.all (fun x => dvd12 x u))
7
8 def sup12 (xs : List Nat) : Option Nat :=
9   let ubs := upperBounds xs
10  ubs.find? (fun s => ubs.all (fun u => dvd12 s u))
11
12 #eval sup12 [4, 6]    -- some 12 (lcm)
13 #eval sup12 [2, 3]    -- some 6
14
15 -- Infimum: greatest lower bound
16 def lowerBounds (xs : List Nat) : List Nat :=
17   D12.filter (fun l => xs.all (fun x => dvd12 l x))
18
19 def inf12 (xs : List Nat) : Option Nat :=
20   let lbs := lowerBounds xs
21   lbs.find? (fun s => lbs.all (fun l => dvd12 l s))
22
23 #eval inf12 [4, 6]    -- some 2 (gcd)

```

```

24 #eval inf12 [3, 4] -- some 1
25
26 -- Print Hasse diagram edges
27 def covers (a b : Nat) : Bool :=
28   dvd12 a b && a != b &&
29   !D12.any (fun c => c != a && c != b && dvd12 a c && dvd12 c b)
30
31 def printHasse : IO Unit := do
32   for a in D12 do
33     for b in D12 do
34       if covers a b then IO.println s! "{a} → {b}"
35
36 #eval printHasse -- 1→2, 1→3, 2→4, 2→6, 3→6, 4→12, 6→12

```

1.6 Exercises

Exercise 1.1 — Reflexivity and symmetry

Show that a relation that is both symmetric and antisymmetric must be a subset of the equality relation, i.e. $a R b \rightarrow a = b$. Prove this in Lean 4.

```

1 theorem symm_antisymm_eq {α : Type} (r : Rel α)
2   (hs : Symmetric r) (ha : Antisymm r) :
3   ∀ a b : α, r a b → a = b := by
4   intro a b h
5   exact ha a b h (hs a b h)

```

Exercise 1.2 — Divisibility on a small set

Consider the divisibility relation restricted to the set $\{1, 2, 3, 4, 6, 12\}$.

- Draw its Hasse diagram.
- Which pairs of elements are incomparable?
- In Lean 4, define a type `D12` with six constructors and prove that divisibility on it satisfies `le_refl`.

Exercise 1.3 — Product order

Given two partial orders $(A, \leq_A)$ and $(B, \leq_B)$, define the *product order* on $A \times B$ by $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq_A a_2 \wedge b_1 \leq_B b_2$. Prove in Lean 4 that this is indeed a partial order by implementing the typeclass instance.

```

1 instance [OrderBook.PartialOrder α] [OrderBook.PartialOrder β] :
2   OrderBook.PartialOrder (α × β) where
3   le p q := PartialOrder.le p.1 q.1 ∧ PartialOrder.le p.2 q.2
4   le_refl := fun ⟨a, b⟩ => ⟨PartialOrder.le_refl a,

```

```

5           PartialOrder.le_refl b)
6 le_antisymm := fun ⟨a1,b1⟩ ⟨a2,b2⟩ ⟨h1,h2⟩ ⟨h3,h4⟩ => by
7   have := PartialOrder.le_antisymm _ _ h1 h3
8   have := PartialOrder.le_antisymm _ _ h2 h4
9   simp_all
10 le_trans := fun ⟨a1,b1⟩ ⟨a2,b2⟩ ⟨a3,b3⟩ ⟨h1,h2⟩ ⟨h3,h4⟩ =>
11   ⟨PartialOrder.le_trans _ _ _ h1 h3,
12     PartialOrder.le_trans _ _ _ h2 h4⟩

```

Exercise 1.4 — Reverse order

If $(A, \leq)$ is a partial order, then so is $(A, \geq)$ where $a \geq b \iff b \leq a$. This is called the *dual* or *opposite* order. Implement a `Dual` wrapper type and its `PartialOrder` instance. We will use this construction crucially when encoding Sudoku domains in Chapter 8.

Chapter 2

Special Elements and Monotone Maps

In a poset, certain elements play privileged roles (least, greatest, bounds), and certain functions are especially well-behaved (monotone maps). This chapter equips us with the vocabulary for both.

2.1 Extremal Elements

Definition — Minimum, Maximum, Minimal, Maximal

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $m \in S$ is the *minimum* of S if $\forall s \in S, m \leq s$.
- $m \in S$ is the *maximum* of S if $\forall s \in S, s \leq m$.
- $m \in S$ is *minimal* in S if $\nexists s \in S, s < m$.
- $m \in S$ is *maximal* in S if $\nexists s \in S, m < s$.

A minimum, if it exists, is unique (by antisymmetry). A minimal element need not be unique. When a poset has a unique minimum we call it $\perp$ (bottom), and a unique maximum $\top$ (top).

The distinction between minimum and minimal is subtle but important. A *minimum* beats every other element; a *minimal* element merely has no element strictly below it.

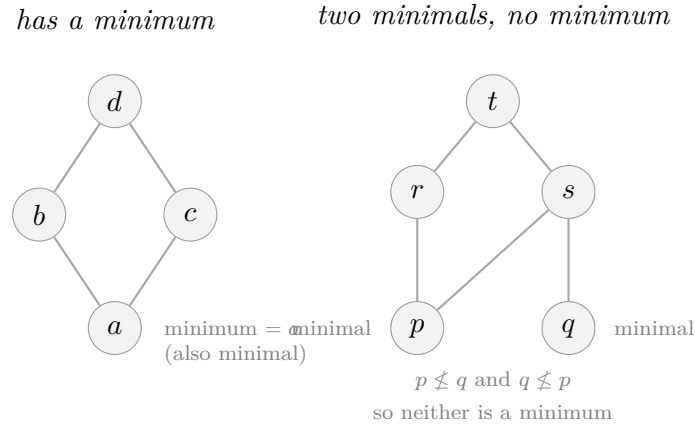


Figure 2.1: Left: a poset with a unique minimum a (every element is above a). Right: a poset where p and q are both minimal (nothing is below either) but there is no minimum (neither $p \leq q$ nor $q \leq p$).

```

1 namespace OrderBook
2
3 -- Bottom and top elements
4 class Bot (α : Type) where bot : α
5 class Top (α : Type) where top : α
6
7 notation "⊥" => Bot.bot
8 notation "⊤" => Top.top
9
10 -- Being a bounded partial order
11 class BoundedPartialOrder (α : Type) extends PartialOrder α, Bot α, Top α
12   ↪ where
13     bot_le : ∀ (a : α), (⊥ : α) ≤ a
14     le_top : ∀ (a : α), a ≤ (⊤ : α)
15 end OrderBook
    
```

Example — Bounded orders

- $(\mathcal{P}(S), \subseteq)$: $\perp = \emptyset$, $\top = S$.
- $(\{0, 1\}, \leq)$: $\perp = 0$, $\top = 1$.
- $(\mathbb{N}, \leq)$: $\perp = 0$, no $\top$ (unbounded above).
- The divisibility poset on $\{1, \dots, 12\}$: $\perp = 1$ (divides everything), no single $\top$.

2.2 Upper and Lower Bounds

Definition — Bounds

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $u \in P$ is an *upper bound* of S if $\forall s \in S, s \leq u$.
- $l \in P$ is a *lower bound* of S if $\forall s \in S, l \leq s$.
- The *supremum* (or *least upper bound*) of S , written $\sup S$, is the smallest upper bound of S when it exists.
- The *infimum* (or *greatest lower bound*) of S , written $\inf S$, is the largest lower bound of S when it exists.

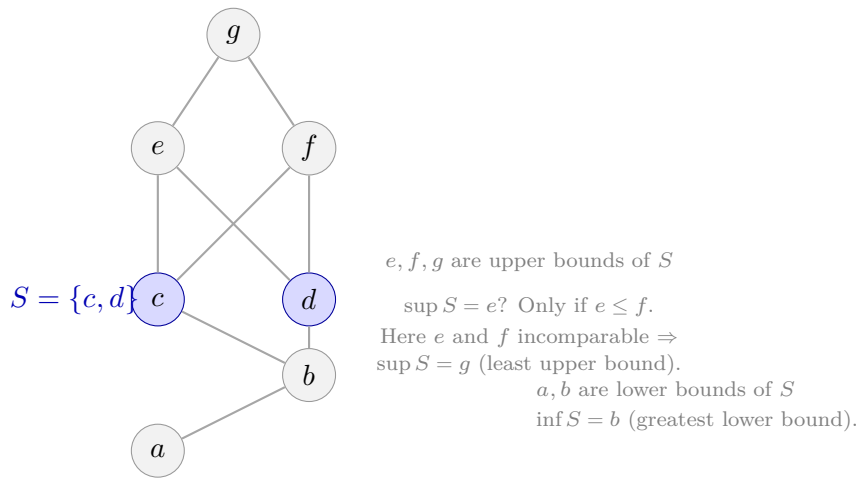


Figure 2.2: Illustration of upper bounds, lower bounds, supremum and infimum for the subset $S = \{c, d\}$ (shaded blue) in a poset. The upper bounds of S are all elements $\geq$ both c and d : here e , f , and g . The *supremum* is the *least* of these; since e and f are incomparable, the least upper bound is g . The lower bounds are a and b ; the infimum is b since $a \leq b$.

Uniqueness of sup/inf

In a poset, if $\sup S$ exists, it is unique. Similarly for $\inf S$.

Proof. Suppose u and v are both least upper bounds of S . Since u is an upper bound and v is a *least* upper bound, $v \leq u$. By symmetry $u \leq v$. Antisymmetry gives $u = v$. $\square$

```

1 -- The supremum of two elements (binary version)
2 -- (We'll generalise to arbitrary subsets in Chapter 4.)
3 structure IsSupOf {α : Type} [OrderBook.PartialOrder α]
4   (a b sup : α) : Prop where
5   ge_a  : a ≤ sup
6   ge_b  : b ≤ sup

```

```

7  least : ∀ (u : α), a ≤ u → b ≤ u → sup ≤ u
8
9  -- Uniqueness of the binary sup
10 theorem sup_unique {α : Type} [OrderBook.PartialOrder α]
11     {a b s t : α}
12     (hs : IsSupOf a b s) (ht : IsSupOf a b t) : s = t := by
13     apply PartialOrder.le_antisymm
14     · exact hs.least t ht.ge_a ht.ge_b
15     · exact ht.least s hs.ge_a hs.ge_b
    
```

2.3 Monotone Maps

Definition — Monotone Map

A function $f : (P, \leq_P) \rightarrow (Q, \leq_Q)$ between posets is *monotone* (or *order-preserving*) if $\forall a, b \in P, a \leq_P b \rightarrow f(a) \leq_Q f(b)$.

Monotone maps are the morphisms of the category of posets. They will be central to propagators: every propagator is a monotone function on a lattice.

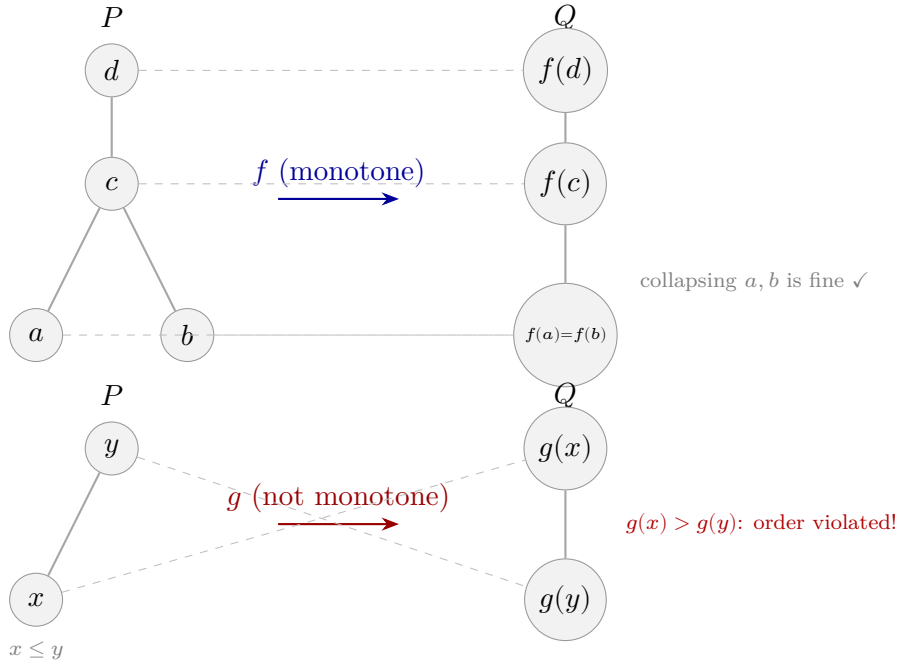


Figure 2.3: Top: a monotone map f preserves the order — whenever $a \leq_P b$ in the source, $f(a) \leq_Q f(b)$ in the target. Here f collapses the incomparable elements a and b to the same image, which is permitted. Bottom: a non-monotone map g where $x \leq y$ but $g(x) > g(y)$, violating the order.

```

2
3 structure Monotone {α β : Type}
4   [PartialOrder α] [PartialOrder β] (f : α → β) : Prop where
5   map_le : ∀ (a b : α), a ≤ b → f a ≤ f b
6
7 -- Composition of monotone maps is monotone
8 theorem Monotone.comp {α β γ : Type}
9   [PartialOrder α] [PartialOrder β] [PartialOrder γ]
10  {f : α → β} {g : β → γ}
11  (hf : Monotone f) (hg : Monotone g) : Monotone (g ∘ f) where
12  map_le := fun a b h => hg.map_le _ _ (hf.map_le _ _ h)
13
14 end OrderBook

```

Example — Monotone functions

- $f(n) = n + 5$ is monotone on $(\mathbb{N}, \leq)$.
- Set union $S \mapsto S \cup \{x\}$ is monotone on $(\mathcal{P}(A), \subseteq)$.
- Set intersection $S \mapsto S \cap T$ is monotone on $(\mathcal{P}(A), \subseteq)$ for fixed T .
- Any constant function $f(x) = c$ is monotone.
- The identity function is always monotone.

2.4 Exercises

Exercise 2.1 — Monotone composition

Give an example showing that the composition of two *antitone* maps (where $a \leq b \rightarrow f(b) \leq f(a)$) is monotone.

Exercise 2.2 — Fixed points

A *fixed point* of $f : P \rightarrow P$ is an element x with $f(x) = x$. Show that if f is monotone and $a \leq f(a)$, then the chain $a \leq f(a) \leq f^2(a) \leq \dots$ is indeed a chain (all elements pairwise comparable with the given order).

Exercise 2.3 — Characterising PartialOrder via Monotone

Show that the identity map $\text{id} : (P, \leq) \rightarrow (P, \leq)$ is always monotone, and prove this in Lean 4 using your `Monotone` structure.

2.5 Computation Interlude: Running Monotone Maps

Building on the computable divisibility order from Chapter 1, we can now compute with monotone maps and verify their properties at runtime.

Checking monotonicity

Given a function on `D12`, we can decide whether it is monotone by checking all pairs.

```

1  -- Check all comparable pairs to see if f preserves the order
2  def isMonotone (f : Nat → Nat) : Bool :=
3    D12.all (fun a =>
4      D12.all (fun b =>
5        -- if a | b then f(a) | f(b)
6        !dvd12 a b || dvd12 (f a) (f b)))
7
8  -- The identity is monotone
9  #eval isMonotone id                -- true
10
11 -- Multiplying by 2: if a | b then 2a | 2b
12 #eval isMonotone (· * 2)           -- true
13
14 -- A non-monotone map: send every element to its "complement" 12/n
15 -- 1↦12, 2↦6, 3↦4, 4↦3, 6↦2, 12↦1 – this reverses the order
16 #eval isMonotone (fun n => 12 / n) -- false (it's antitone)
17
18 -- But composing it with itself gives monotone again
19 #eval isMonotone (fun n => 12 / (12 / n)) -- true

```

Computing fixed points

A fixed point of f is an element where $f(x) = x$. We can find them all.

```

1  def fixedPoints (f : Nat → Nat) : List Nat :=
2    D12.filter (fun x => f x == x)
3
4  #eval fixedPoints id                -- [1, 2, 3, 4, 6, 12]
5  #eval fixedPoints (fun _ => 1)      -- [1]
6  #eval D12.filter (fun n => 12 / n == n) -- [] (no fixed points for n ↦ 12/n)

```

Iterating a monotone map

From Exercise 2.2, we know that if f is monotone and $a \leq f(a)$, the chain $a, f(a), f^2(a), \dots$ is ascending. We can watch it converge.

```

1  -- Apply f repeatedly until fixpoint or step limit
2  def iterToFixpoint (f : Nat → Nat) (start : Nat) (limit : Nat) : List Nat :=
3    let rec go (x : Nat) (n : Nat) (acc : List Nat) : List Nat :=
4      if n = 0 then acc.reverse
5      else let y := f x
6        if y == x then (acc.reverse ++ [x])
7        else go y (n - 1) (x :: acc)

```

```
8   go start limit []
9
10  -- f(n) = if 2*n ∈ D12 then 2*n else n
11  def double12 (n : Nat) : Nat :=
12    if D12.contains (n * 2) then n * 2 else n
13
14  #eval iterToFixpoint double12 1 10   -- [1, 2, 4] ascending chain!
15  #eval iterToFixpoint double12 3 10   -- [3, 6, 12]
16  #eval iterToFixpoint double12 12 10  -- [12] already a fixed point
```

The output directly illustrates the ascending chain idea: each application of `double12` moves strictly upward in the divisibility order until it hits a fixed point. Note *which* fixed point: starting from 1 the chain stops at 4, because $8 \notin D_{12}$ makes 4 a fixed point of `double12` — a monotone map can have many fixed points, and iteration finds the one above where you started. Starting from 3 instead, the chain climbs all the way to the top element 12.

Chapter 3

Lattices

A poset in which every pair of elements has both a supremum and an infimum is called a *lattice*. This apparently modest requirement turns out to yield a rich algebraic structure that shows up in logic, topology, computer science, and — as we will see in Part II — concurrency and constraint solving.

3.1 Meet and Join

Definition — Lattice

A *lattice* is a poset $(L, \leq)$ in which every two elements $a, b \in L$ have:

- a *meet* (greatest lower bound) $a \sqcap b$, and
- a *join* (least upper bound) $a \sqcup b$.

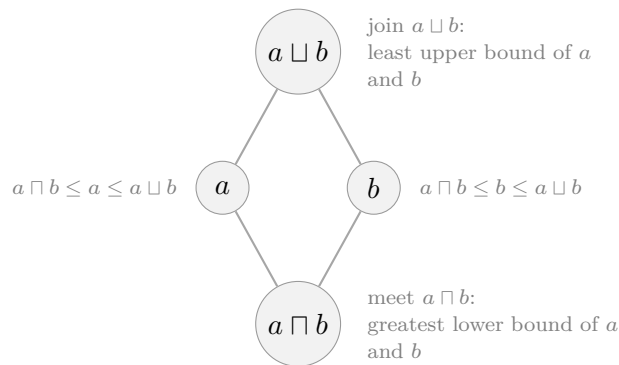


Figure 3.1: The meet $a \sqcap b$ sits below both a and b ; the join $a \sqcup b$ sits above both. In a lattice every pair of elements has both.

The meet and join satisfy the following axioms (which can be taken as an equivalent *algebraic* definition of a lattice):

$a \sqcap b = b \sqcap a$	$a \sqcup b = b \sqcup a$	(commutativity)
$(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c)$	$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c)$	(associativity)
$a \sqcap a = a$	$a \sqcup a = a$	(idempotence)
$a \sqcap (a \sqcup b) = a$	$a \sqcup (a \sqcap b) = a$	(absorption)

The order relation can be recovered from meet or join:

$$a \leq b \iff a \sqcap b = a \iff a \sqcup b = b.$$

In Lean 4:

```
1 namespace OrderBook
2
3 class Lattice (α : Type) extends PartialOrder α where
4   inf : α → α → α    -- meet (∩)
5   sup : α → α → α    -- join (∪)
6   -- meet is a lower bound
7   inf_le_left  : ∀ (a b : α), inf a b ≤ a
8   inf_le_right : ∀ (a b : α), inf a b ≤ b
9   -- meet is the greatest lower bound
10  le_inf : ∀ (a b c : α), a ≤ b → a ≤ c → a ≤ inf b c
11  -- join is an upper bound
12  sup_le_left  : ∀ (a b : α), a ≤ sup a b
13  sup_le_right : ∀ (a b : α), b ≤ sup a b
14  -- join is the least upper bound
15  le_sup : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
16
17 -- Notation
18 infixl:70 " ∩ " => Lattice.inf
19 infixl:65 " ∪ " => Lattice.sup
20
21 end OrderBook
```

3.1.1 Derived Properties

We can immediately derive commutativity and idempotence from the axioms:

```
1 namespace OrderBook
2 open Lattice PartialOrder
3
4 variable {α : Type} [Lattice α]
5
```



```

6  -- Commutativity of meet
7  theorem inf_comm (a b :  $\alpha$ ) : a  $\sqcap$  b = b  $\sqcap$  a := by
8    apply le_antisymm
9    · apply le_inf
10     · exact inf_le_right a b
11     · exact inf_le_left a b
12    · apply le_inf
13     · exact inf_le_right b a
14     · exact inf_le_left b a
15
16  -- Idempotence of meet
17  theorem inf_idem (a :  $\alpha$ ) : a  $\sqcap$  a = a := by
18    apply le_antisymm
19    · exact inf_le_left a a
20    · exact le_inf a a a (le_refl a) (le_refl a)
21
22  -- Commutativity of join
23  theorem sup_comm (a b :  $\alpha$ ) : a  $\sqcup$  b = b  $\sqcup$  a := by
24    apply le_antisymm
25    · apply le_sup
26     · exact sup_le_right b a
27     · exact sup_le_left b a
28    · apply le_sup
29     · exact sup_le_right a b
30     · exact sup_le_left a b
31
32  -- Absorption: a  $\sqcup$  (a  $\sqcap$  b) = a
33  theorem sup_inf_self (a b :  $\alpha$ ) : a  $\sqcup$  (a  $\sqcap$  b) = a := by
34    apply le_antisymm
35    · apply le_sup
36     · exact le_refl a
37     · exact inf_le_left a b
38    · exact sup_le_left a (a  $\sqcap$  b)
39
40 end OrderBook

```

3.2 The Duality Principle

Every lattice has a *dual*: swap $\sqcap \leftrightarrow \sqcup$ and $\leq \leftrightarrow \geq$. Any theorem proved in a lattice has a dual theorem for free.

Duality Principle

If φ is a theorem about lattices, then so is the statement obtained by replacing every occurrence of $\sqcap$ with $\sqcup$, $\sqcup$ with $\sqcap$, $\leq$ with $\geq$, $\perp$ with $\top$, and $\top$ with $\perp$.

```

1 structure Dual ( $\alpha$  : Type) where
2   val :  $\alpha$ 
3
4   -- Step 1: flip  $\leq$ .  $a \leq_{\text{dual}} b$  means  $b.\text{val} \leq_{\text{orig}} a.\text{val}$ .
5 instance { $\alpha$  : Type} [LE  $\alpha$ ] : LE (Dual  $\alpha$ ) where
6   le a b := LE.le ( $\alpha$  :=  $\alpha$ ) b.val a.val
7
8   -- Step 2: the dual partial order. Our PartialOrder class carries its
9   -- own `le` field, so the flipped order is stated again here (the LE
10  -- instance above only supplies notation).
11 open OrderBook in
12 instance { $\alpha$  : Type} [PartialOrder  $\alpha$ ] : PartialOrder (Dual  $\alpha$ ) where
13   le a b := PartialOrder.le ( $\alpha$  :=  $\alpha$ ) b.val a.val
14   le_refl a := PartialOrder.le_refl ( $\alpha$  :=  $\alpha$ ) a.val
15   le_antisymm a b h1 h2 := by
16     cases a; cases b; congr 1
17     exact PartialOrder.le_antisymm ( $\alpha$  :=  $\alpha$ ) _ _ h2 h1
18   le_trans a b c h1 h2 := by
19     cases a; cases b; cases c
20     exact PartialOrder.le_trans ( $\alpha$  :=  $\alpha$ ) _ _ _ h2 h1
21
22   -- Step 3: the dual lattice swaps inf and sup.
23 open OrderBook in
24 instance { $\alpha$  : Type} [Lattice  $\alpha$ ] : Lattice (Dual  $\alpha$ ) where
25   inf      := fun a b => (Lattice.sup ( $\alpha$  :=  $\alpha$ ) a.val b.val)
26   sup      := fun a b => (Lattice.inf ( $\alpha$  :=  $\alpha$ ) a.val b.val)
27   inf_le_left := fun a b => Lattice.sup_le_left ( $\alpha$  :=  $\alpha$ ) a.val b.val
28   inf_le_right := fun a b => Lattice.sup_le_right ( $\alpha$  :=  $\alpha$ ) a.val b.val
29   sup_le_left := fun a b => Lattice.inf_le_left ( $\alpha$  :=  $\alpha$ ) a.val b.val
30   sup_le_right := fun a b => Lattice.inf_le_right ( $\alpha$  :=  $\alpha$ ) a.val b.val
31   le_inf := fun a b c h1 h2 => by
32     change Lattice.sup ( $\alpha$  :=  $\alpha$ ) b.val c.val  $\leq$  a.val
33     exact Lattice.le_sup ( $\alpha$  :=  $\alpha$ ) b.val c.val a.val h1 h2
34   le_sup := fun a b c h1 h2 => by
35     change LE.le ( $\alpha$  :=  $\alpha$ ) c.val (Lattice.inf ( $\alpha$  :=  $\alpha$ ) a.val b.val)
36     exact Lattice.le_inf ( $\alpha$  :=  $\alpha$ ) c.val a.val b.val h1 h2

```

structure vs **def**. Using **structure** `Dual` rather than **def** `Dual α := α` makes `Dual α` and `α` genuinely distinct types. With a transparent **def**, Lean’s elaborator sometimes picks up the wrong LE instance for hypotheses, causing type mismatches even when the terms are definitionally equal. The **structure** version forces explicit `.val` unwrapping and `<...>` rewrapping, which is more verbose but unambiguous.

Why cases in `le_antisymm` and `le_trans`? `cases a` destructs `Dual.mk v` into the underlying `v : α`, so subsequent lemmas see plain `α` inequalities rather than `Dual`-typed ones. In `le_antisymm`, `congr 1` further reduces `Dual.mk x = Dual.mk y` to `x = y`.

Why `change` in `le_inf` and `le_sup`? After destructuring `<a> <b> <c>`, the hypotheses `h1` and `h2` are still elaborated under the dual LE. `change` restates the goal entirely in terms of `α`’s operations, giving Lean enough information to match the original lemma’s type.

Why `inf` and `sup swap` is the whole point: turning the Hasse diagram upside down exchanges every meet with every join.

3.3 Important Lattice Examples

3.3.1 The Two-Element Lattice 2

```

1 -- Bool as a lattice: false < true.
2 -- Core's Bool.le_antisymm / Bool.le_trans take their element
3 -- arguments implicitly, so we eta-expand them to match our class
4 -- fields, which declare the elements explicitly.
5 instance : OrderBook.Lattice Bool where
6   le      := (· ≤ ·)    -- Bool has a built-in LE in core Lean 4
7   le_refl := Bool.le_refl
8   le_antisymm := fun a b => Bool.le_antisymm
9   le_trans  := fun a b c => Bool.le_trans
10  inf      := (· && ·)
11  sup      := (· || ·)
12  inf_le_left  := by decide
13  inf_le_right := by decide
14  le_inf       := by decide
15  sup_le_left  := by decide
16  sup_le_right := by decide
17  le_sup       := by decide

```

The **by decide** tactic works here because `Bool` is a finite type and all the statements are decidable.

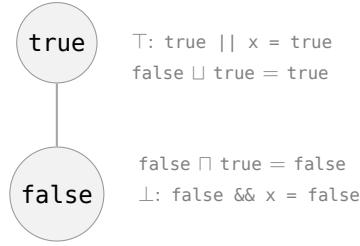


Figure 3.2: The two-element lattice $\mathbf{2}$ on `Bool`. Meet is `&&`, join is `||`.

3.3.2 The Powerset Lattice

```

1  -- Sets ( $\alpha \rightarrow \text{Prop}$ ) form a lattice under subset inclusion
2  def Set.inter { $\alpha$  : Type} (s t : Set  $\alpha$ ) : Set  $\alpha$  := fun a => s a  $\wedge$  t a
3  def Set.union { $\alpha$  : Type} (s t : Set  $\alpha$ ) : Set  $\alpha$  := fun a => s a  $\vee$  t a
4
5  instance { $\alpha$  : Type} : OrderBook.Lattice (Set  $\alpha$ ) where
6    le      := Set.subset
7    le_refl := fun s a ha => ha
8    le_antisymm := fun s t h1 h2 => funext fun a => propext (h1 a, h2 a)
9    le_trans := fun s t u h1 h2 a ha => h2 a (h1 a ha)
10   inf      := Set.inter
11   sup      := Set.union
12   inf_le_left  := fun s t a (hs, _) => hs
13   inf_le_right := fun s t a (_, ht) => ht
14   le_inf       := fun s t u hst hsu a ha => (hst a ha, hsu a ha)
15   sup_le_left  := fun s t a hs => Or.inl hs
16   sup_le_right := fun s t a ht => Or.inr ht
17   le_sup       := fun s t u hsu ht u a h =>
18     h.elim (hsu a) (htu a)
    
```

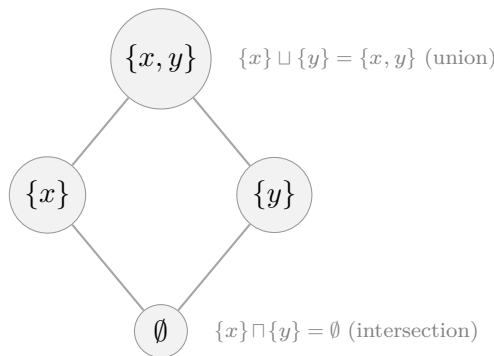


Figure 3.3: The powerset lattice $\mathcal{P}(\{x, y\})$. Meet is $\cap$, join is $\cup$. $\{x\}$ and $\{y\}$ are incomparable.

3.3.3 The Divisibility Lattice on Divisors of n

For divisors of a fixed natural number n , divisibility is a lattice where gcd is the meet and lcm is the join.

```

1  -- On natural numbers: gcd is meet, lcm is join under divisibility order
2  instance : OrderBook.Lattice Nat where
3    le      := (· | ·)  -- divisibility
4    le_refl := Nat.dvd_refl
5    le_antisymm := fun a b => Nat.dvd_antisymm
6    le_trans := fun a b c => Nat.dvd_trans
7    inf     := Nat.gcd
8    sup     := Nat.lcm
9    inf_le_left  := Nat.gcd_dvd_left
10   inf_le_right := Nat.gcd_dvd_right
11   le_inf      := fun a b c => Nat.dvd_gcd
12   sup_le_left  := Nat.dvd_lcm_left
13   sup_le_right := Nat.dvd_lcm_right
14   le_sup      := fun a b c ha hb => Nat.lcm_dvd ha hb

```

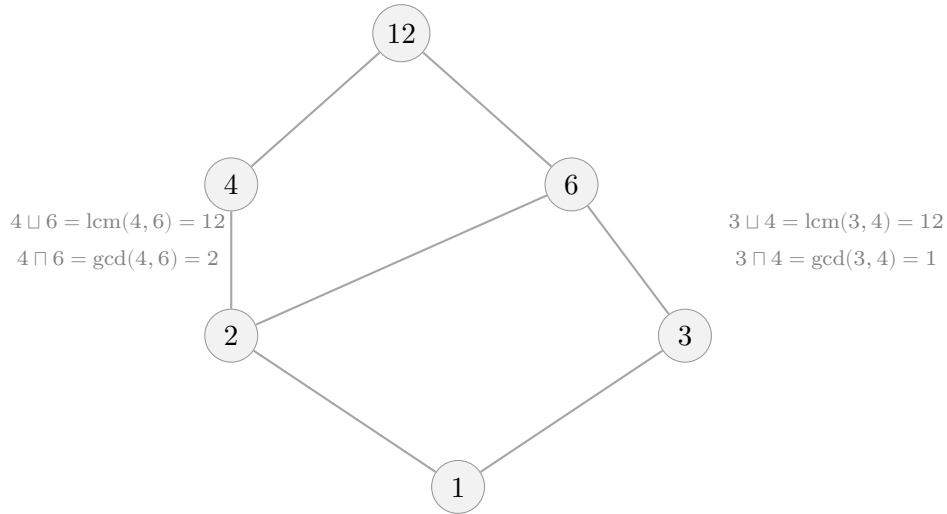


Figure 3.4: The divisibility lattice on divisors of 12. Meet is gcd, join is lcm. Every incomparable pair still has a gcd and lcm in the set, confirming this is a lattice.

3.4 Distributive Lattices

Not every lattice satisfies the distributive law; those that do are especially well-behaved.

Definition — Distributive Lattice

A lattice is *distributive* if $a \sqcap (b \sqcup c) = (a \sqcap b) \sqcup (a \sqcap c)$ (equivalently, the dual law holds too).

The powerset lattice and Boolean lattices are distributive. Figure 3.5 shows the two smallest non-distributive lattices.

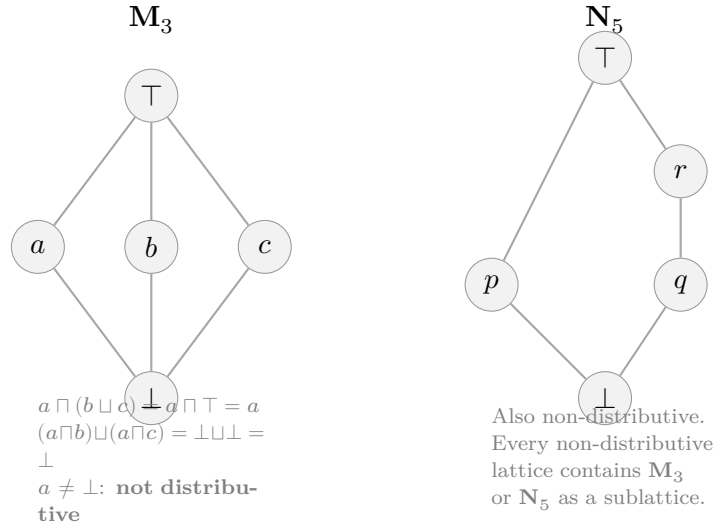


Figure 3.5: The two smallest non-distributive lattices. $\mathbf{M}_3$ (left) has three mutually incomparable middle elements; the distributive law fails because the meet does not distribute over the join. $\mathbf{N}_5$ (right) is the pentagon lattice. A classical theorem states that a lattice is distributive if and only if it contains neither as a sublattice.

3.5 Exercises

Exercise 3.1 — Ordering intervals by information

Define the type `Interval` of pairs $[l, h]$ with $l, h : \mathbb{Z}$ and $l \leq h$. Define the *information order*: $[l_1, h_1] \leq [l_2, h_2]$ iff $l_1 \leq l_2$ and $h_2 \leq h_1$ — that is, the second interval is *contained* in the first, so narrower = more informative = higher. Prove that this is a partial order in Lean 4. Then investigate: which pairs of intervals have a meet? A join? (*Careful*: the meet — the smallest interval containing both — always exists, but two *disjoint* intervals have no join in this type, because there is no “empty interval” to represent the contradiction. Chapter 7 completes the picture by adding one, together with unbounded ends.)

Exercise 3.2 — Absorb, then join

Prove in Lean 4 that in any lattice: $a \sqcap (a \sqcup b) = a$ (absorption).

Exercise 3.3 — Characterising the order

Prove the equivalence $a \leq b \iff a \sqcup b = b$ in Lean 4, using only the lattice axioms.

```

1 open OrderBook in
2 theorem le_iff_sup_eq {α : Type} [OrderBook.Lattice α] (a b : α) :
3   a ≤ b ↔ a ⊔ b = b := by
4   constructor
5   · intro h
6     apply PartialOrder.le_antisymm
7     · exact Lattice.le_sup a b b h (PartialOrder.le_refl b)
8     · exact Lattice.sup_le_right a b
9   · intro h
10    have : b = a ⊔ b := h.symm
11    rw [this]
12    exact Lattice.sup_le_left a b

```

Exercise 3.4 — The product lattice

Given two lattices $[Lattice\ \alpha]$ and $[Lattice\ \beta]$, define a $Lattice\ (\alpha \times \beta)$ instance with the component-wise order: $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq a_2 \wedge b_1 \leq b_2$, and component-wise meet and join: $(a_1, b_1) \sqcap (a_2, b_2) = (a_1 \sqcap a_2, b_1 \sqcap b_2)$, $(a_1, b_1) \sqcup (a_2, b_2) = (a_1 \sqcup a_2, b_1 \sqcup b_2)$. Prove all eight lattice axioms. (*Hint*: each axiom reduces to the corresponding axiom applied component-wise.)

Chapter 4

Complete Lattices and the Fixed-Point Theorem

Lattices let us take meets and joins of *pairs* of elements. A *complete* lattice extends this to arbitrary (possibly infinite) subsets. This extra power buys us one of the most useful theorems in theoretical computer science: the Knaster–Tarski fixed-point theorem.

4.1 Complete Lattices

Definition — Complete Lattice

A lattice $(L, \leq)$ is *complete* if every subset $S \subseteq L$ has a supremum $\bigvee S$ and an infimum $\bigwedge S$ in L .

In particular, the empty set has a supremum (which becomes $\perp$) and infimum (which becomes $\top$), so every complete lattice is bounded. Taking $S = L$ gives $\bigvee L = \top$ and $\bigwedge L = \perp$.

We encode complete lattices using a predicate representing a subset:

```
1 namespace OrderBook
2
3 -- We represent a "subset" as a predicate  $\alpha \rightarrow \text{Prop}$ .
4 -- This is the same definition as `Set  $\alpha$ ` from Chapter 1
5 -- (`def Set ( $\alpha : \text{Type}$ ) : Type :=  $\alpha \rightarrow \text{Prop}$ `).
6 -- We use `abbrev` rather than `def` so that Lean treats `Pred  $\alpha$ `
7 -- as completely transparent – unfolding it automatically during
8 -- elaboration without needing `simp` or `unfold`. This makes the
9 -- `sSup`/`sInf` signatures below easier to work with.
10 -- `def Set` was opaque because we wanted `Set  $\alpha$ ` to have its own
11 -- identity for attaching instances; here we just want a readable alias.
12 abbrev Pred ( $\alpha : \text{Type}$ ) :=  $\alpha \rightarrow \text{Prop}$ 
13
14 class CompleteLattice ( $\alpha : \text{Type}$ ) extends Lattice  $\alpha$  where
15   sSup : Pred  $\alpha \rightarrow \alpha$     -- supremum of a set (a "big  $\cup$ ")
```

```

16 sInf : Pred α → α    -- infimum of a set    (a "big n")
17 le_sSup : ∀ (s : Pred α) (a : α), s a → a ≤ sSup s
18 sSup_le : ∀ (s : Pred α) (u : α), (∀ a, s a → a ≤ u) → sSup s ≤ u
19 sInf_le : ∀ (s : Pred α) (a : α), s a → sInf s ≤ a
20 le_sInf : ∀ (s : Pred α) (l : α), (∀ a, s a → l ≤ a) → l ≤ sInf s
21
22 end OrderBook

```

The powerset lattice is the canonical example of a complete lattice: $\bigvee \mathcal{F} = \bigcup \mathcal{F}$ and $\bigwedge \mathcal{F} = \bigcap \mathcal{F}$.

```

1 -- Set α is a complete lattice
2 instance {α : Type} : OrderBook.CompleteLattice (Set α) where
3   -- (inheriting the Lattice instance from Chapter 3)
4   sSup := fun F a => ∃ s : Set α, F s ∧ s a    -- union of a family
5   sInf := fun F a => ∀ s : Set α, F s → s a    -- intersection of a family
6   le_sSup := fun F s hs a ha => ⟨s, hs, ha⟩
7   sSup_le := fun F u hu a ⟨s, hs, ha⟩ => hu s hs a ha
8   sInf_le := fun F s hs a ha => ha s hs
9   le_sInf := fun F l hl a ha s hs => hl s hs a ha
10  -- (le, inf, sup fields inherited)
11  le      := Set.subset
12  le_refl := fun s a h => h
13  le_antisymm := fun s t h1 h2 => funext fun a => propext ⟨h1 a, h2 a⟩
14  le_trans  := fun s t u h1 h2 a ha => h2 a (h1 a ha)
15  inf       := Set.inter; sup := Set.union
16  inf_le_left := fun s t a ⟨hs, _⟩ => hs
17  inf_le_right := fun s t a ⟨_, ht⟩ => ht
18  le_inf      := fun s t u hst hsu a ha => ⟨hst a ha, hsu a ha⟩
19  sup_le_left := fun s t a hs => Or.inl hs
20  sup_le_right := fun s t a ht => Or.inr ht
21  le_sup      := fun s t u hsu ht u a h => h.elim (hsu a) (htu a)

```

4.2 Why Fixed Points Matter

A *fixed point* of $f : L \rightarrow L$ is an element x where $f(x) = x$. This is where an iterative process *stabilises*: no further application of f changes anything.

- **Propagator networks.** Cell values stop changing when the propagators reach a fixed point of “apply all propagators once”. That stable state is the least fixed point — the most conservative set of conclusions forced by the constraints, nothing more.
- **Dataflow analysis.** A compiler iterates a flow function until it stabilises. The result is the least fixed point of that function.

- **Denotational semantics.** The meaning of a recursive program is the least fixed point of a semantic functional.
- **Regular expressions.** The set matched by a^* is the least fixed point of $S \mapsto \{\varepsilon\} \cup a \cdot S$.

We will return to the propagator connection in Chapter 6, where network execution terminates at the least fixed point of the combined propagation function.

4.3 The Knaster–Tarski Fixed-Point Theorem

Knaster–Tarski (1955)

Let $(L, \leq)$ be a complete lattice and $f : L \rightarrow L$ a monotone function. Then f has a *least fixed point* $\mu f = \bigwedge \{x \mid f(x) \leq x\}$ and a *greatest fixed point* $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Proof. Let $P = \{x \in L \mid f(x) \leq x\}$ (the set of *pre-fixed points*) and let $m = \bigwedge P$.

Step 1: $f(m) \leq m$. For every $x \in P$ we have $m \leq x$, so $f(m) \leq f(x) \leq x$ by monotonicity. Hence $f(m)$ is a lower bound of P , so $f(m) \leq m$.

Step 2: $m \leq f(m)$. From Step 1, $f(f(m)) \leq f(m)$ (applying f to $f(m) \leq m$ and using monotonicity gives $f(f(m)) \leq f(m)$, so $f(m) \in P$). Since m is the infimum of P , we have $m \leq f(m)$.

Conclusion: $f(m) = m$, so m is a fixed point. Any other fixed point x^* satisfies $f(x^*) = x^* \leq x^*$, so $x^* \in P$, hence $m \leq x^*$. So m is the *least* fixed point. $\square$

```

1 namespace OrderBook
2 open CompleteLattice PartialOrder
3
4 -- Knaster-Tarski: least fixed point of a monotone function
5 -- on a complete lattice
6 theorem knaster_tarski {α : Type} [CompleteLattice α]
7   (f : α → α) (hf : Monotone f) :
8     ∃ (lfp : α), f lfp = lfp ∧
9       ∀ (x : α), f x = x → lfp ≤ x := by
10   -- Define P = {x | f(x) ≤ x} and m = ⋂ P
11   let P : Pred α := fun x => f x ≤ x
12   let m := sInf P
13   -- Step 1: f(m) ≤ m
14   have step1 : f m ≤ m := by
15     apply le_sInf
16     intro x hx
17     -- m ≤ x, so f(m) ≤ f(x) ≤ x. (The class states le_trans with
18     -- explicit element arguments, hence the placeholders `__`.)

```

```

19   exact le_trans _ _ _ (hf.map_le m x (sInf_le P x hx)) hx
20   -- Step 2:  $m \leq f(m)$  ( $f(m) \in P$  since  $f(f(m)) \leq f(m)$ )
21   have step2 :  $m \leq f\ m$  := by
22     apply sInf_le
23     exact hf.map_le _ _ step1
24   -- Conclude  $m$  is a fixed point
25   have fixed :  $f\ m = m$  := le_antisymm _ _ step1 step2
26   refine ⟨m, fixed, ?_⟩
27   -- Minimality: every fixed point is a pre-fixed point, so  $m \leq x$ .
28   intro x hx
29   apply sInf_le
30   show  $f\ x \leq x$ 
31   rw [hx]
32   exact le_refl x
33 end OrderBook
    
```

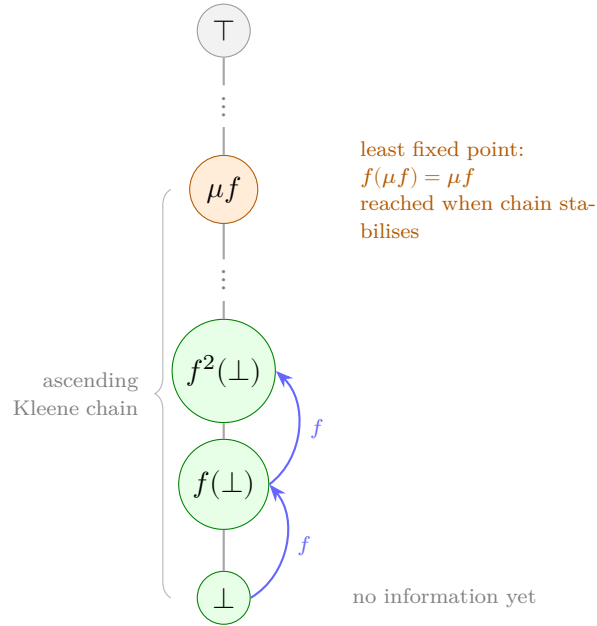


Figure 4.1: The ascending Kleene chain starting from $\perp$ under a monotone function f . Each application of f moves upward. If the lattice satisfies the ascending chain condition the sequence stabilises at the least fixed point μf .

4.4 Chains and Convergence

Chains are the lattice-theoretic version of sequences in calculus, and understanding them is essential for understanding why iterative computations terminate.

Definition — Chain

A *chain* in a poset $(P, \leq)$ is a subset $C \subseteq P$ where every pair of elements is comparable: for all $a, b \in C$, either $a \leq b$ or $b \leq a$. An *ascending chain* is a chain written as a sequence $a_0 \leq a_1 \leq a_2 \leq \dots$.

The parallel with calculus sequences is direct:

Calculus	Lattice theory
Sequence $a_0 \leq a_1 \leq a_2 \leq \dots$	Ascending chain
Limit $\lim_{n \rightarrow \infty} a_n$	Supremum $\bigsqcup_n a_n$
Convergence	Chain stabilising
Completeness of $\mathbb{R}$	Completeness of the lattice
Cauchy sequence	Ascending chain condition

Just as the completeness of $\mathbb{R}$ guarantees that every bounded ascending sequence has a limit, the completeness of a lattice guarantees that every chain has a supremum. Without completeness, the supremum might not exist in your structure — exactly as $\sqrt{2}$ does not exist in $\mathbb{Q}$ even though there are rational sequences converging to it.

The Ascending Chain Condition

A poset satisfies the *ascending chain condition* (ACC) if every strictly ascending chain $a_0 < a_1 < a_2 < \dots$ is finite — there are no infinite strictly ascending sequences.

This is the lattice analogue of a sequence that reaches its limit in finite steps rather than merely approaching it. When the ACC holds, the Kleene chain

$$\perp \leq f(\perp) \leq f^2(\perp) \leq \dots$$

must eventually stabilise at some $f^n(\perp) = f^{n+1}(\perp)$, giving a fixed point in finite iterations.

Example — ACC in practice

The flat lattice **FlatNat** that we will build in Chapter 6 ($\perp$ = unknown, exact values in the middle, $\top$ = contradiction) satisfies the ACC: the longest strictly ascending chains are $\perp < \text{known}(n) < \text{conflict}$, of length 3. So any propagator network using **FlatNat** cells terminates after at most two updates per cell.

The interval lattice of Chapter 7 (where narrower intervals sit *higher*, and the ends may be unbounded) does *not* satisfy the ACC: $(-\infty, +\infty) < [0, +\infty) < [1, +\infty) < [2, +\infty) < \dots$ is an infinite strictly ascending chain. This is why interval propagators in general need a *widening* operator to force termination — although, as we will see, well-behaved constraint systems often reach a fixed point on their own.

$\perp$ is the starting point, not the limit

A common point of confusion: the supremum of the Kleene chain is *not* $\perp$. $\perp$ is the *minimum* of the chain — the smallest element in it. The supremum is the smallest element above the whole chain, which is wherever the iteration converges.

In the calculus analogy: for the sequence $0, \frac{1}{2}, \frac{3}{4}, \frac{7}{8}, \dots$ converging to 1, the minimum of the sequence is 0 but the supremum (limit) is 1. The starting point and the limit are different objects. The Kleene chain behaves the same way: $\perp$ is where you start, μf is where you end up.

4.5 Exercises

Exercise 4.1 — Greatest fixed point

Using the Knaster–Tarski proof as a template, prove the existence of the *greatest* fixed point $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Exercise 4.2 — Ascending Kleene chain

For a complete lattice with a bottom element $\perp$ and a monotone f , the sequence $\perp \leq f(\perp) \leq f^2(\perp) \leq \dots$ is called the *ascending Kleene chain*. If the lattice satisfies the *ascending chain condition* (every ascending chain eventually stabilises), this chain reaches μf in finitely many steps. Prove in Lean 4 that each element of this sequence is $\leq$ the next.

Part II

Propagator Networks

Chapter 5

The Propagator Model

“A propagator network is a collection of machines that compute values for cells by combining information from other cells.”

— G.J. Sussman & A. Radul, *The Art of the Propagator*, 2009

We now have all the mathematical machinery we need. This chapter explains the propagator model from first principles and makes precise why lattices are not merely convenient but *necessary*.

5.1 Cells: Accumulating Partial Information

The central insight of propagator networks is a shift in perspective:

Traditional cell: stores a **single value**

↓ *evolution*

Propagator cell: accumulates **partial information**

A cell does not *store a value*; it stores *everything we know so far* about a value. New information can arrive from multiple sources and be merged in. The key requirement is that information can only *increase*: once we know something, we cannot un-know it.

This is precisely the structure of a *join-semilattice with a bottom element*: the bottom represents “we know nothing”, and merging two pieces of information is the join ($\sqcup$).

Definition — Information Order

A *bounded join-semilattice* is a partial order $(J, \leq)$ equipped with a join $\sqcup$ and a bottom $\perp$ such that:

- $a \sqcup b$ is the least upper bound of a and b ,
- $\perp \leq a$ for all a ,
- $a \leq b$ is read as “ b contains at least as much information as a ”.

We call this the *information order*.

```
1 namespace Propagators
2
3 class BoundedJoinSemilattice (α : Type) extends OrderBook.PartialOrder α where
```

```

4  sup :  $\alpha \rightarrow \alpha \rightarrow \alpha$ 
5  bot :  $\alpha$ 
6  le_sup_left  :  $\forall (a\ b : \alpha), a \leq \text{sup } a\ b$ 
7  le_sup_right :  $\forall (a\ b : \alpha), b \leq \text{sup } a\ b$ 
8  sup_le      :  $\forall (a\ b\ c : \alpha), a \leq c \rightarrow b \leq c \rightarrow \text{sup } a\ b \leq c$ 
9  bot_le      :  $\forall (a : \alpha), \text{bot} \leq a$ 
10
11 -- Convenient notation
12 infixl:65 "  $\sqcup$  " => BoundedJoinSemilattice.sup
13 notation   " $\perp$ "  => BoundedJoinSemilattice.bot
14
15 end Propagators

```

5.2 Why Information Can Only Increase

Suppose a propagator computes that “ x is between 3 and 7”. Later another propagator deduces that “ x is between 5 and 12”. Combining these we know “ x is between 5 and 7”. Notice: the combined knowledge is *strictly more specific* than either input.

If we allowed information to *decrease* (e.g., forgetting the $[3, 7]$ constraint and keeping only $[5, 12]$), we would lose soundness: our cell might claim values that the original propagators had ruled out.

Monotonicity Ensures Soundness

In a propagator network over a bounded join-semilattice, if every propagator is monotone, then the network is *sound*: the final cell values are consistent with all constraints.

Intuitively: since each step can only add information (move up in the lattice), the final state contains at least as much information as any intermediate state. Nothing is lost; nothing false is added (assuming correct propagators).

5.3 Propagators as Monotone Functions

Definition — Propagator

Given cells with values in $(J, \leq)$, a *propagator* is a monotone function $f : J^n \rightarrow J$ (reading from n input cells and writing to one output cell by merging the result).

Writing is done by merging: the output cell c receives $c \leftarrow c \sqcup f(\text{inputs})$. Since $\sqcup$ is the least upper bound, this can only increase c .

```

1 namespace Propagators
2

```

```

3  -- A unary propagator: a pure function with a bundled monotonicity proof.
4  -- The condition  $\text{fn } a \leq \text{fn } b$  is the standard order-theoretic definition.
5  -- (Equivalent to BoundedJoinSemilattice.sup (fn a) (fn b) = fn b,
6  -- but the  $\leq$  form is cleaner to state and prove.)
7  structure Propagator1 ( $\alpha \beta$  : Type)
8      [BoundedJoinSemilattice  $\alpha$ ]
9      [BoundedJoinSemilattice  $\beta$ ] where
10     fn      :  $\alpha \rightarrow \beta$ 
11     monotone :  $\forall (a \ b : \alpha), a \leq b \rightarrow \text{fn } a \leq \text{fn } b$ 
12
13  -- A binary propagator: monotone in both arguments.
14  structure Propagator2 ( $\alpha \beta \gamma$  : Type)
15      [BoundedJoinSemilattice  $\alpha$ ]
16      [BoundedJoinSemilattice  $\beta$ ]
17      [BoundedJoinSemilattice  $\gamma$ ] where
18     fn      :  $\alpha \rightarrow \beta \rightarrow \gamma$ 
19     monotone :  $\forall (a1 \ a2 : \alpha) (b1 \ b2 : \beta),$ 
20          $a1 \leq a2 \rightarrow b1 \leq b2 \rightarrow \text{fn } a1 \ b1 \leq \text{fn } a2 \ b2$ 
21
22  end Propagators

```

Specification vs. implementation. `Propagator1` and `Propagator2` capture what a propagator *is* mathematically: a function with a bundled monotonicity proof. They are the *specification*.

In practice we use `PropStep = IO Bool`: a plain `IO` action that reads cells, computes, writes, and returns whether anything changed. There is no bundled proof — the user is trusted to write monotone propagators. Section 6.5 below proves that `addProp` and `subProp` do in fact satisfy the `Propagator2` specification. For the capstone projects the proofs would be analogous but verbose, so we omit them.

5.4 Networks, Scheduling, and Quiescence

A *propagator network* is a directed graph where nodes are cells and edges are propagators. Execution proceeds as follows:

1. Initialise all cells to $\perp$.
2. Place all propagators on a *work queue*.
3. While the queue is non-empty:
 - (a) Take a propagator p from the queue.
 - (b) Compute the new value $v = p(\text{input cells})$.
 - (c) Merge v into the output cell: $c \leftarrow c \sqcup v$.
 - (d) If c changed, add all propagators that read from c back to the queue.

4. The network has reached *quiescence* when the queue is empty.

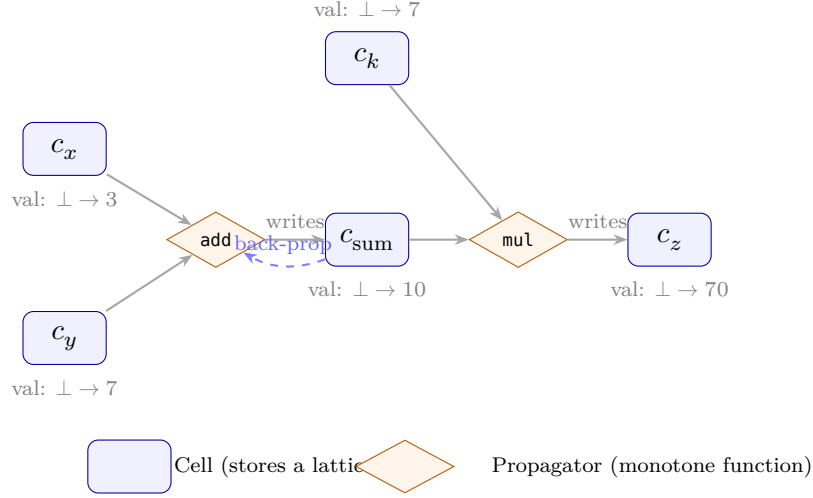


Figure 5.1: A small propagator network computing $x + y = \text{sum}$ and $\text{sum} \times k = z$ (with $x = 3$, $y = 7$, $k = 7$). Cells hold lattice values initialised to $\perp$. Propagators read from inputs and merge results into output cells via $\sqcup$. The dashed arrow shows back-propagation: when c_{sum} is updated the **add** propagator is re-queued.

Convergence

If the lattice satisfies the *ascending chain condition* (every strictly ascending sequence is finite), then the network reaches quiescence in finite time.

This is immediate: each cell value is non-decreasing and the chain condition guarantees it cannot increase indefinitely.

For the Sudoku capstone the domain lattice is finite, so convergence is guaranteed outright; for the type-inference capstone each program generates finitely many constraints whose refinements stabilise. The interval lattice of Chapter 7 is the cautionary tale: it fails the ACC, and we will be careful to say why its example nevertheless terminates.

5.5 The Fundamental Role of the Lattice

Let us be precise about why we need a lattice rather than merely some arbitrary type.

- **Join ($\sqcup$) for merging.** When two propagators independently derive information about the same cell, we need to combine their results. The join gives the *least informative* value that is *at least as informative* as both. No information is lost; no information is invented.
- **Bottom ($\perp$) for initialisation.** Before any propagator has run, a cell contains *no information*. The bottom element represents this “empty” state.
- **Partial order ($\leq$) for monitoring change.** We must know whether a cell’s value actually changed after merging, to decide whether to re-schedule dependent

propagators. The partial order provides this: if $c \sqcup v = c$ (the new value adds no information), nothing changed.

- **Antisymmetry for termination.** If a cell's value can only go up and the lattice is finite, the antisymmetry of $\leq$ ensures each value is visited at most once.

Any type that fails to be a join-semilattice with $\perp$ is simply not a valid cell domain for a propagator network. The lattice structure is not a convenience — it is a prerequisite.

Chapter 6

Building a Propagator Network in Lean 4

Theory in hand, we now build a complete, runnable propagator network in Lean 4, starting from our typeclass definitions and ending with a working scheduler.

6.1 Interlude: Imperative Lean in Half a Page

Everything so far has been pure functions and proofs. Propagator networks, however, are stateful machines — cells get updated in place — so from here on the code looks imperative. Lean makes this look like any mainstream language via **do**-notation:

```
1 def imperativeTour : IO Unit := do
2   let r ← IO.mkRef 0           -- a mutable reference holding a Nat
3   let mut best := 0            -- a local mutable variable
4   for i in [0:5] do             -- i runs over 0, 1, 2, 3, 4
5     r.modify (· + i)           -- add i to the reference's contents
6     if i > best then best := i
7   let total ← r.get             -- read the reference back
8   IO.println s!"total = {total}, best = {best}"
9
10 #eval imperativeTour          -- total = 10, best = 4
```

Four things to know, in decreasing order of importance:

- **do sequences effects.** A **do** block runs in a *monad* — here **IO**, the type of actions that may touch the outside world. **let x ← action** runs the action and binds its result; plain **let x := e** is an ordinary pure binding.
- **IO.Ref α** is a mutable memory cell holding an α , with **IO.mkRef**, **.get**, **.set**, and **.modify**. Our propagator cells wrap exactly one of these.
- **mut, for, while, if** inside **do** are sugar. The compiler rewrites them into folds and recursive calls with the mutable variables threaded through as accumulators — no actual mutation of **best** ever happens. The full desugaring story is in the primer of Appendix A.
- **Id.run** lets pure code use the same sugar. Running a **do** block in the **do**

nothing monad `Id` and unwrapping it with `Id.run` gives imperative-looking code with a pure result. The Sudoku solver in Chapter 8 uses this heavily.

That is all the imperative Lean this book needs.

6.2 Mutable Cells

Lean 4 has an `IO` monad with mutable references (`IO.Ref`). We use these to implement cells.

```

1 namespace Propagators
2
3 -- A Cell wraps an IO.Ref and a type with a BoundedJoinSemilattice
4 structure Cell (α : Type) where
5   ref      : IO.Ref α
6
7 variable {α : Type} [BoundedJoinSemilattice α] [DecidableEq α]
8
9 -- Create a new cell initialised to ⊥
10 def Cell.new : IO (Cell α) := do
11   let r ← IO.mkRef (BoundedJoinSemilattice.bot (α := α))
12   return { ref := r }
13
14 -- Read the current value
15 def Cell.read (c : Cell α) : IO α := c.ref.get
16
17 -- Attempt to write new information.
18 -- Returns true if the cell's value increased.
19 def Cell.write (c : Cell α) (v : α) : IO Bool := do
20   let old ← c.ref.get
21   let merged := old ⊔ v
22   if merged == old then
23     return false      -- no new information
24   else
25     c.ref.set merged
26     return true       -- cell updated, re-schedule dependents
27
28 end Propagators

```

We require `DecidableEq α` to compare `merged == old`. For the finite lattices we use in our capstone projects this is always available.

6.3 The Network and Scheduler

```

1 namespace Propagators
2
3 -- A propagator step is an IO action that returns
4 -- true if it caused any cell to change.
5 abbrev PropStep := IO Bool
6
7 -- Run a list of propagators to fixpoint.
8 -- Each pass re-runs all propagators; we stop when
9 -- a full pass causes no changes.
10 def runToFixpoint (steps : Array PropStep) : IO Unit := do
11   let mut changed := true
12   while changed do
13     changed := false
14     for step in steps do
15       let c ← step
16       if c then changed := true
17
18 -- A smarter scheduler that only re-runs propagators
19 -- whose inputs changed (using a dependency queue).
20 structure Network where
21   steps      : Array PropStep
22   -- In a production implementation, steps would be indexed
23   -- and cells would hold lists of dependent propagator IDs.
24   -- For clarity we use the simple "run all" scheduler here.
25
26 def Network.run (n : Network) : IO Unit :=
27   runToFixpoint n.steps
28
29 end Propagators

```

6.4 Building Propagators: Numeric Example

Let us define the *flat natural number lattice*: information is either “unknown” ($\perp$), a specific number, or “contradiction” ($\top$).

```

1 namespace Propagators
2
3 inductive FlatNat where
4   | unknown      : FlatNat      --  $\perp$ : no information
5   | known (n : Nat) : FlatNat    -- exactly  $n$ 
6   | conflict     : FlatNat      --  $\top$ : contradiction
7

```

```

8  -- We give the order a NAME rather than an anonymous match inside the
9  -- instance: later proofs can then unfold it with `simp [FlatNat.le]`,
10 -- and the rfl-lemma `le_def` below lets simp see through `≤`.
11 def FlatNat.le (a b : FlatNat) : Prop :=
12   match a, b with
13   | .unknown, _      => True      -- unknown ≤ everything
14   | _, .conflict     => True      -- everything ≤ conflict
15   | .known m, .known n => m = n  -- known values only ≤ themselves
16   | .conflict, .unknown => False
17   | .conflict, .known _ => False
18   | .known _, .unknown => False
19
20 instance : OrderBook.PartialOrder FlatNat where
21   le := FlatNat.le
22   le_refl := by intro a; cases a <;> simp [FlatNat.le]
23   le_antisymm := by
24     intro a b hab hba
25     cases a <;> cases b <;> simp_all [FlatNat.le]
26   le_trans := by
27     intro a b c hab hbc
28     cases a <;> cases b <;> cases c <;> simp_all [FlatNat.le]
29
30 theorem FlatNat.le_def (a b : FlatNat) : (a ≤ b) = FlatNat.le a b := rfl
31
32 instance : BoundedJoinSemilattice FlatNat where
33   le := FlatNat.le
34   le_refl := OrderBook.PartialOrder.le_refl
35   le_antisymm := OrderBook.PartialOrder.le_antisymm
36   le_trans := OrderBook.PartialOrder.le_trans
37   bot := .unknown
38   sup a b := match a, b with
39   | .unknown, x      => x
40   | x, .unknown      => x
41   | .known m, .known n => if m = n then .known m else .conflict
42   | _, _            => .conflict
43   bot_le := by intro a; cases a <;> simp [FlatNat.le_def, FlatNat.le]
44   le_sup_left := by
45     intro a b
46     cases a <;> cases b <;> simp_all [FlatNat.le_def, FlatNat.le]
47     rename_i m n
48     by_cases h : m = n <;> simp [FlatNat.le_def, FlatNat.le, h]
49   le_sup_right := by
50     intro a b
51     cases a <;> cases b <;> simp_all [FlatNat.le_def, FlatNat.le]
52     rename_i m n
53     by_cases h : m = n <;> simp [FlatNat.le_def, FlatNat.le, h]
54   -- The known/known case needs its own `if m = n` split at the end.

```

```

55   sup_le      := by
56     intro a b c hac hbc
57     cases a <;> cases b <;> cases c <;> simp_all [FlatNat.le_def, FlatNat.le]
58     rename_i m n
59     by_cases h : m = n <;> simp [FlatNat.le, h]
60
61 deriving instance DecidableEq for FlatNat
62
63 end Propagators

```

6.5 Monotonicity Proofs for addProp and subProp

The PropStep actions `addProp` and `subProp` implement pure functions on `FlatNat`. We can extract those functions and prove they satisfy the `Propagator2` specification.

Why conflict propagates. There is a subtlety: the pure function must return `conflict` (not `unknown`) when an input is `conflict`, otherwise monotonicity fails. Suppose we returned `unknown` for conflicted inputs. We have `known 3 ≤ conflict` in `FlatNat`, so monotonicity would demand `addFn (known 3) (known 2) ≤ addFn conflict (known 2)` — but the left side is `known 5`, the right side would be `unknown`, and `known 5 <math>\not\leq</math> unknown. Returning conflict fixes this: known 5 ≤ conflict always holds.`

```

1 namespace Propagators
2
3 -- Pure addition: both inputs known → known sum.
4 -- Either input conflicted → conflict propagates.
5 -- Otherwise → unknown (no information yet).
6 def FlatNat.addFn : FlatNat → FlatNat → FlatNat
7   | .known a, .known b => .known (a + b)
8   | .conflict, _      => .conflict
9   | _, .conflict      => .conflict
10  | _, _              => .unknown
11
12 -- Pure subtraction: known total - known a, with conflict on underflow.
13 def FlatNat.subFn (s x : FlatNat) : FlatNat :=
14   match s, x with
15   | .known total, .known a =>
16     if total ≥ a then .known (total - a) else .conflict
17   | .conflict, _ | _, .conflict => .conflict
18   | _, _                      => .unknown
19
20 -- Proof: addFn is monotone.
21 -- Strategy: case-split all 81 constructor combinations (3 × 3 × 3 × 3).
22 -- Most cases are trivial:
23 -- • unknown ≤ anything → addFn unknown ... = unknown ≤ anything.

```

```

24 -- • anything ≤ conflict → addFn ... conflict = conflict, and anything ≤
    ↪ conflict.
25 -- • known m ≤ unknown is False → hypothesis contradiction closes the
    ↪ goal.
26 -- The only non-trivial case is known/known on both sides, where the
27 -- hypotheses force the arguments (and hence both sides) to be equal.
28 -- simp_all with the unfolded order closes every case.
29 def addPropagator : Propagator2 FlatNat FlatNat FlatNat where
30   fn      := FlatNat.addFn
31   monotone := by
32     intro a1 a2 b1 b2 ha hb
33     cases a1 <=> cases a2 <=> cases b1 <=> cases b2
34     <=> simp_all [FlatNat.addFn, FlatNat.le_def, FlatNat.le]
35
36 -- Proof: subFn is monotone.
37 -- Same strategy. After the case split, the remaining goals are stuck
38 -- on the `if total ≥ a` scrutinees; `split <=> simp_all` finishes them.
39 def subPropagator : Propagator2 FlatNat FlatNat FlatNat where
40   fn      := FlatNat.subFn
41   monotone := by
42     intro a1 a2 b1 b2 ha hb
43     cases a1 <=> cases a2 <=> cases b1 <=> cases b2
44     <=> simp_all [FlatNat.subFn, FlatNat.le_def, FlatNat.le]
45     <=> split <=> simp_all
46
47 -- Connection to PropStep actions:
48 -- addProp cx cy csum (§6.6) is addPropagator.fn wrapped in IO:
49 -- it reads cx and cy, computes addFn, and merges into csum.
50 -- subProp csum cx cy wraps subPropagator.fn similarly.
51 -- The Propagator2 instances certify that these IO actions are sound.
52
53 end Propagators

```

6.6 A Complete Running Example

Example — Arithmetic propagation

We know: $x + y = 10$ and $x = 3$. A propagator should deduce $y = 7$.

```

1 namespace Propagators
2
3 open FlatNat
4
5 -- Addition propagator: addFn wrapped in IO – read the inputs, apply
6 -- the pure monotone function, merge the result into the output cell.

```

```

7  -- When either input is unknown, addFn returns unknown = 1, and
8  -- merging 1 changes nothing (write returns false). When an input is
9  -- conflict, addFn returns conflict – so contradictions propagate.
10 def addProp (cx cy csum : Cell FlatNat) : PropStep := do
11   let x ← cx.read
12   let y ← cy.read
13   csum.write (FlatNat.addFn x y)
14
15  -- Subtraction (used to deduce y = sum - x): subFn wrapped in IO.
16 def subProp (csum cx cy : Cell FlatNat) : PropStep := do
17   let s ← csum.read
18   let x ← cx.read
19   cy.write (FlatNat.subFn s x)
20
21 def exampleNetwork : IO Unit := do
22   let cx ← Cell.new (α := FlatNat)
23   let cy ← Cell.new (α := FlatNat)
24   let csum ← Cell.new (α := FlatNat)
25
26   -- Seed known facts
27   let _ ← cx.write (.known 3)
28   let _ ← csum.write (.known 10)
29
30   let net : Network := {
31     steps := #[
32       addProp cx cy csum,
33       subProp csum cx cy,
34       subProp csum cy cx
35     ]
36   }
37
38   net.run
39
40   let yVal ← cy.read
41   match yVal with
42   | .known n => IO.println s!"y = {n}"    -- prints: y = 7
43   | .unknown => IO.println "y is unknown"
44   | .conflict => IO.println "contradiction!"
45
46 end Propagators

```

6.7 Why the Lattice Is Load-Bearing Here

Notice several things:

- `Cell.write` can be called multiple times from different propagators without coordination. The join ensures the final value is consistent with all of them.
- Running `addProp` when either input is `unknown` correctly does nothing: `addFn` returns $\perp$, merging $\perp$ into the target changes nothing, and `write` returns `false` — the network simply waits for more information.
- The `conflict` element propagates correctly: if two propagators derive incompatible values, the join produces `conflict`, which then propagates to any cell that depends on this one.
- The whole system terminates because `FlatNat` has height 2: after at most two increases, any cell stabilises.

6.8 Exercises

Exercise 6.1 — Three-cell constraint

Add a `mulProp` propagator for $x \times y = z$ with the `FlatNat` lattice. Build a network with $x \times y = 12$, $x = 3$, and verify it deduces $y = 4$.

Exercise 6.2 — Conflict detection

Seed a network with $x = 3$ from one source and $x = 5$ from another. Verify that `cx.read` returns `.conflict`.

Chapter 7

The Interval Lattice

Flat lattices are powerful but coarse: a cell either knows the exact value or knows nothing. *Interval lattices* let us represent partial numeric knowledge as a range, enabling *bound propagation* — the engine behind many real-world constraint solvers.

7.1 Intervals as Partial Information

An interval $[l, h]$ says: “the true value lies between l and h ”. The wider the interval, the less we know. The narrowest possible interval is a single point $[n, n]$, meaning we know the value exactly.

Chapter 5’s convention therefore dictates the order: more information = *higher*, so narrower intervals must sit *above* wider ones, and the join $\sqcup$ — “merge what two sources tell us” — must be the *intersection*: if one source says $x \in [3, 7]$ and another says $x \in [5, 12]$, together they say $x \in [5, 7]$.

Two boundary cases complete the picture, and both should look familiar from **FlatNat**:

- $\perp$ must mean “no information at all”. For intervals that is “the value could be any integer”: the unbounded interval $(-\infty, +\infty)$. This forces us to make $\pm\infty$ representable.
- $\top$ must mean “contradiction”. Intersect two disjoint intervals — $[0, 1] \sqcup [5, 6]$ — and *no* value remains: the *empty* interval plays exactly the role of **conflict**.

Definition — Interval Lattice

Let the domain be the extended integers $\mathbb{Z}_\infty = \mathbb{Z} \cup \{-\infty, +\infty\}$. Define:

- An *interval* is either $[l, h]$ with $l \leq h$ (where l may be $-\infty$ and h may be $+\infty$), or the *empty interval* $\emptyset$.
- The *information order*: $[l_1, h_1] \leq [l_2, h_2] \iff l_1 \leq l_2 \wedge h_2 \leq h_1$ (the second interval is contained in the first: narrower = more informative = higher), and $I \leq \emptyset$ for every interval I .
- $\perp = (-\infty, +\infty)$: no information. $\top = \emptyset$: contradiction — matching **FlatNat**, where contradiction is also the *top*.
- The *join* (merge two pieces of information) is *intersection*: $[l_1, h_1] \sqcup [l_2, h_2] = [\max(l_1, l_2), \min(h_1, h_2)]$, or $\emptyset$ if those bounds cross.

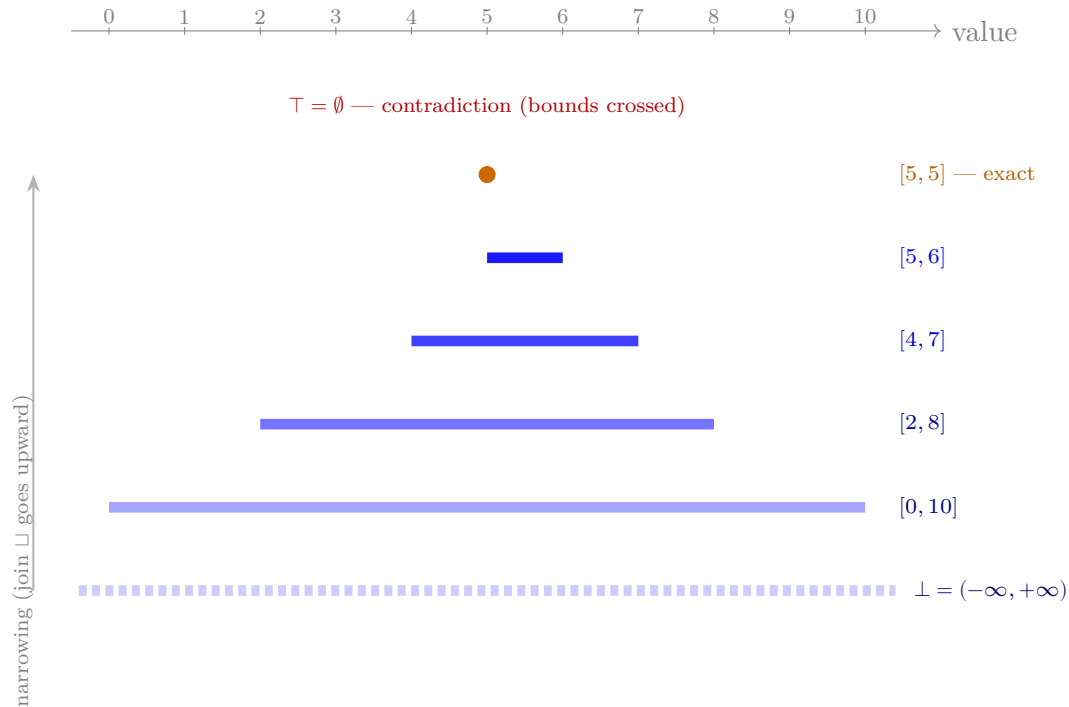


Figure 7.1: The interval information order. $\perp = (-\infty, +\infty)$ (“the value could be anything”) sits at the bottom. Going upward, intervals *narrow*: the join $\sqcup$ merges two constraints by intersecting them, which is at least as narrow as either input. At the very top sits $\top = \emptyset$, the empty interval: two constraints with no common value — a contradiction.

7.2 Bounds: Option Int as the Extended Integers

How do we represent $\pm\infty$? The cheapest honest encoding in Lean is `Option Int`: some n is the finite bound n , and `none` means “no bound at all” — read it as $-\infty$ when it appears as a lower bound and $+\infty$ when it appears as an upper bound. (`Option` is Lean’s “maybe” type; if you know Rust’s `Option` or Haskell’s `Maybe`, this is the same thing.)

```
1 namespace Intervals
2
3 -- A bound on an integer. `none` encodes "no bound at all":
4 -- read it as  $-\infty$  when it is a lower bound and  $+\infty$  when it is an
5 -- upper bound.
6 abbrev Bound := Option Int
7
8 -- Order on LOWER bounds: none =  $-\infty$  is the loosest lower bound.
9 def loLe : Bound → Bound → Prop
10 | none, _      => True
11 | some _, none => False
12 | some a, some b => a ≤ b
```



```

13
14 -- Order on UPPER bounds: none = +∞ is the loosest upper bound.
15 def hiLe : Bound → Bound → Prop
16 | _,      none    => True
17 | none,   some _ => False
18 | some a, some b => a ≤ b
19
20 -- A lower bound and an upper bound are consistent unless both are
21 -- finite and crossed (lo > hi).
22 def ordered : Bound → Bound → Prop
23 | some l, some h => l ≤ h
24 | _,      _      => True
25
26 instance : (l h : Bound) → Decidable (ordered l h)
27 | none,   none    => isTrue True.intro
28 | none,   some _ => isTrue True.intro
29 | some _, none    => isTrue True.intro
30 | some l, some h => inferInstanceAs (Decidable (l ≤ h))
31
32 -- The tighter of two lower bounds (their max, with none = -∞) ...
33 def loMax : Bound → Bound → Bound
34 | none, b => b
35 | a, none => a
36 | some a, some b => some (max a b)
37
38 -- ... and the tighter of two upper bounds (their min, with none = +∞).
39 def hiMin : Bound → Bound → Bound
40 | none, b => b
41 | a, none => a
42 | some a, some b => some (min a b)
43
44 end Intervals

```

Note that we needed *two* order relations: on lower bounds `none` is the smallest thing there is; on upper bounds it is the largest. Keeping the two roles in separate definitions (`loLe` vs. `hiLe`, `loMax` vs. `hiMin`) keeps every later proof a brainless case split.

Everything we need about bounds follows from a dozen tiny lemmas. Each is proved the same way: `cases` on every bound in sight (an `Option` has just two constructors), then `simp` to unfold the definitions, then `omega` for whatever integer arithmetic remains.

```

1 namespace Intervals
2
3 -- Small lemmas about bounds. Each is a mechanical case split.
4 theorem loLe_refl (a : Bound) : loLe a a := by
5   cases a <|> simp [loLe]
6

```

```

7 theorem hiLe_refl (a : Bound) : hiLe a a := by
8   cases a <;> simp [hiLe]
9
10 theorem loLe_trans {a b c : Bound} : loLe a b → loLe b c → loLe a c := by
11   cases a <;> cases b <;> cases c <;> simp [loLe] <;> omega
12
13 theorem hiLe_trans {a b c : Bound} : hiLe a b → hiLe b c → hiLe a c := by
14   cases a <;> cases b <;> cases c <;> simp [hiLe] <;> omega
15
16 theorem loLe_antisymm {a b : Bound} : loLe a b → loLe b a → a = b := by
17   cases a <;> cases b <;> simp [loLe] <;> omega
18
19 theorem hiLe_antisymm {a b : Bound} : hiLe a b → hiLe b a → a = b := by
20   cases a <;> cases b <;> simp [hiLe] <;> omega
21
22 theorem loLe_max_left (a b : Bound) : loLe a (loMax a b) := by
23   cases a <;> cases b <;> simp [loLe, loMax] <;> omega
24
25 theorem loLe_max_right (a b : Bound) : loLe b (loMax a b) := by
26   cases a <;> cases b <;> simp [loLe, loMax] <;> omega
27
28 theorem loMax_le {a b c : Bound} :
29   loLe a c → loLe b c → loLe (loMax a b) c := by
30   cases a <;> cases b <;> cases c <;> simp [loLe, loMax] <;> omega
31
32 theorem hiMin_le_left (a b : Bound) : hiLe (hiMin a b) a := by
33   cases a <;> cases b <;> simp [hiLe, hiMin] <;> omega
34
35 theorem hiMin_le_right (a b : Bound) : hiLe (hiMin a b) b := by
36   cases a <;> cases b <;> simp [hiLe, hiMin] <;> omega
37
38 theorem le_hiMin {a b c : Bound} :
39   hiLe c a → hiLe c b → hiLe c (hiMin a b) := by
40   cases a <;> cases b <;> cases c <;> simp [hiLe, hiMin] <;> omega
41
42 -- If [l', h'] is a genuine interval squeezed between l and h,
43 -- then l and h cannot be crossed either.
44 theorem ordered_squeeze {l l' h' h : Bound} :
45   loLe l l' → ordered l' h' → hiLe h' h → ordered l h := by
46   cases l <;> cases l' <;> cases h' <;> cases h <;>
47     simp [loLe, hiLe, ordered] <;> omega
48
49 end Intervals

```

7.3 The Interval Lattice in Lean

An interval is either a pair of bounds — carrying a proof that they are not crossed — or the distinguished contradiction `empty`:

```

1 namespace Intervals
2
3 inductive Interval where
4   -- All integers between lo and hi (inclusive; none = unbounded).
5   -- The proof field rules out crossed bounds like [8, 2].
6   | range (lo hi : Bound) (ok : ordered lo hi) : Interval
7   -- T: no value is possible – a contradiction.
8   | empty : Interval
9
10  -- 1: the interval  $(-\infty, +\infty)$  – “the value could be any integer”.
11 def Interval.unbounded : Interval := .range none none True.intro
12
13 -- The interval  $[l, h]$  with finite ends.
14 def Interval.between (l h : Int) (ok : l ≤ h := by omega) : Interval :=
15   .range (some l) (some h) ok
16
17 -- Information order: narrower = more informative = HIGHER.
18 -- b sits above a exactly when b's bounds are at least as tight.
19 def Interval.le (a b : Interval) : Prop :=
20   match a, b with
21   | _, .empty          => True    -- T is above everything
22   | .empty, _          => False
23   | .range l1 h1 _, .range l2 h2 _ => loLe l1 l2 ∧ hiLe h2 h1
24
25 -- Join = INTERSECTION: merge two constraints by keeping the tighter
26 -- bound on each side. If the bounds cross, the constraints are
27 -- contradictory and the join is T = empty.
28 def Interval.sup (a b : Interval) : Interval :=
29   match a, b with
30   | .empty, _ | _, .empty => .empty
31   | .range l1 h1 _, .range l2 h2 _ =>
32     if ok : ordered (loMax l1 l2) (hiMin h1 h2)
33     then .range (loMax l1 l2) (hiMin h1 h2) ok
34     else .empty
35
36 end Intervals

```

Why does the `range` constructor carry a proof? Without the invariant $l \leq h$, “crossed” pairs like `[8, 2]` would be legal values distinct from `empty` — a swarm of extra contradiction elements. Worse, the join of two disjoint intervals would have to be one of those crossed pairs (not `empty`!) to remain a *least* upper bound, and the lattice laws would

quietly break. The proof field makes the illegal states unrepresentable, and (as you will see in `sup_le_thm`) is precisely the fact that rescues the “bounds crossed” case of the join laws. In `Interval.between`, the proof is an *autoparam*: `(ok : l ≤ h := by omega)` tells Lean to run `omega` to fill in the argument whenever the call site omits it, so numeric literals just work: `Interval.between 2 8`.

The partial-order laws are stated as standalone lemmas and then plugged into the instance (the `_thm` suffix avoids confusing a lemma with the instance field it fills):

```

1 namespace Intervals
2
3 theorem Interval.le_refl_thm : ∀ (a : Interval), Interval.le a a := by
4   intro a
5   cases a with
6   | empty => exact True.intro
7   | range l h ok => exact (loLe_refl l, hiLe_refl h)
8
9 theorem Interval.le_antisymm_thm :
10   ∀ (a b : Interval), Interval.le a b → Interval.le b a → a = b := by
11   intro a b hab hba
12   cases a with
13   | empty =>
14     cases b with
15     | empty => rfl
16     | range l h ok => exact absurd hab (fun h => h)
17   | range l1 h1 ok1 =>
18     cases b with
19     | empty => exact absurd hba (fun h => h)
20     | range l2 h2 ok2 =>
21       have hab' : loLe l1 l2 ∧ hiLe h2 h1 := hab
22       have hba' : loLe l2 l1 ∧ hiLe h1 h2 := hba
23       have h1 : l1 = l2 := loLe_antisymm hab'.1 hba'.1
24       have h2 : h1 = h2 := hiLe_antisymm hba'.2 hab'.2
25       subst h1; subst h2
26       rfl -- the `ok` proofs agree by proof irrelevance
27
28 theorem Interval.le_trans_thm :
29   ∀ (a b c : Interval), Interval.le a b → Interval.le b c → Interval.le a c
30   ↔ := by
31   intro a b c hab hbc
32   cases c with
33   | empty => cases a <|> exact True.intro
34   | range l3 h3 ok3 =>
35     cases b with
36     | empty => exact absurd hbc (fun h => h)
37     | range l2 h2 ok2 =>
38       cases a with

```

```

38   | empty => exact absurd hab (fun h => h)
39   | range l1 h1 ok1 =>
40     have hab' : loLe l1 l2 ∧ hiLe h2 h1 := hab
41     have hbc' : loLe l2 l3 ∧ hiLe h3 h2 := hbc
42     exact ⟨loLe_trans hab'.1 hbc'.1, hiLe_trans hbc'.2 hab'.2⟩
43
44 instance : OrderBook.PartialOrder Interval where
45   le      := Interval.le
46   le_refl := Interval.le_refl_thm
47   le_antisymm := Interval.le_antisymm_thm
48   le_trans := Interval.le_trans_thm
49
50 end Intervals

```

Three idioms are worth pausing on. First, lines like `have hab' : loLe l1 l2 ∧ hiLe h2 h1 := hab` do no work at all: once both arguments are `ranges`, `Interval.le a b` *computes* to that conjunction, and the `have` merely re-states the hypothesis at its unfolded type so that `.1/.2` projections apply. Second, `exact absurd hab (fun h => h)` dispatches impossible cases: there `hab`'s type computes to `False`, so the identity function *is* its refutation. Third, antisymmetry ends with plain `rfl` even though the two `ok` proof fields are different terms — proofs of the same proposition are definitionally equal in Lean (*proof irrelevance*).

Now the join laws. The one interesting moment is the “bounds crossed” branch of `sup_le_thm`, which is impossible precisely because the upper bound `c` is itself a well-formed interval squeezed between the merged bounds — this is `ordered_squeeze` doing its job:

```

1 namespace Intervals
2
3 theorem Interval.bot_le_thm :
4   ∀ (a : Interval), Interval.le Interval.unbounded a := by
5   intro a
6   cases a with
7   | empty => exact True.intro
8   | range l h ok => exact ⟨True.intro, True.intro⟩
9
10 theorem Interval.le_sup_left_thm :
11   ∀ (a b : Interval), Interval.le a (Interval.sup a b) := by
12   intro a b
13   cases a with
14   | empty => cases b <|> exact True.intro
15   | range l1 h1 ok1 =>
16     cases b with
17     | empty => exact True.intro
18     | range l2 h2 ok2 =>

```

```

19     simp only [Interval.sup]
20     split
21     · exact ⟨loLe_max_left l1 l2, hiMin_le_left h1 h2⟩
22     · exact True.intro
23
24 theorem Interval.le_sup_right_thm :
25     ∀ (a b : Interval), Interval.le b (Interval.sup a b) := by
26     intro a b
27     cases a with
28     | empty => cases b <=> exact True.intro
29     | range l1 h1 ok1 =>
30         cases b with
31         | empty => exact True.intro
32         | range l2 h2 ok2 =>
33             simp only [Interval.sup]
34             split
35             · exact ⟨loLe_max_right l1 l2, hiMin_le_right h1 h2⟩
36             · exact True.intro
37
38 theorem Interval.sup_le_thm :
39     ∀ (a b c : Interval),
40     Interval.le a c → Interval.le b c → Interval.le (Interval.sup a b) c :=
41     ⇔ by
42     intro a b c hac hbc
43     cases a with
44     | empty => cases c <=> exact hac
45     | range l1 h1 ok1 =>
46         cases b with
47         | empty => cases c <=> exact hbc
48         | range l2 h2 ok2 =>
49             cases c with
50             | empty =>
51                 simp only [Interval.sup]
52                 split <=> exact True.intro
53             | range l3 h3 ok3 =>
54                 have hac' : loLe l1 l3 ∧ hiLe h3 h1 := hac
55                 have hbc' : loLe l2 l3 ∧ hiLe h3 h2 := hbc
56                 have hlo : loLe (loMax l1 l2) l3 := loMax_le hac'.1 hbc'.1
57                 have hhi : hiLe h3 (hiMin h1 h2) := le_hiMin hac'.2 hbc'.2
58                 simp only [Interval.sup]
59                 split
60                 · exact ⟨hlo, hhi⟩
61                 · -- the "crossed bounds" branch is impossible: c is a genuine
62                   -- interval squeezed between the merged bounds.
63                   rename_i hbad
64                   exact absurd (ordered_squeeze hlo ok3 hhi) hbad

```

```

65 instance : Propagators.BoundedJoinSemilattice Interval where
66   le           := Interval.le
67   le_refl      := Interval.le_refl_thm
68   le_antisymm  := Interval.le_antisymm_thm
69   le_trans     := Interval.le_trans_thm
70   bot          := Interval.unbounded
71   sup          := Interval.sup
72   bot_le       := Interval.bot_le_thm
73   le_sup_left  := Interval.le_sup_left_thm
74   le_sup_right := Interval.le_sup_right_thm
75   sup_le       := Interval.sup_le_thm
76
77 -- DecidableEq, needed by Cell.write. Two `range`s are equal iff their
78 -- bounds are; the `ok` proofs never disagree (proof irrelevance).
79 instance : DecidableEq Interval := fun a b =>
80   match a, b with
81   | .empty, .empty => isTrue rfl
82   | .empty, .range _ _ _ => isFalse (fun h => Interval.noConfusion h)
83   | .range _ _ _, .empty => isFalse (fun h => Interval.noConfusion h)
84   | .range l1 h1 _, .range l2 h2 _ =>
85     if h : l1 = l2 ∧ h1 = h2 then
86       isTrue (by obtain ⟨rfl, rfl⟩ := h; rfl)
87     else
88       isFalse (fun heq => by cases heq; exact h ⟨rfl, rfl⟩)
89
90 end Intervals

```

7.4 Interval Arithmetic Propagators

To use `Interval` with the `Cell` infrastructure from Chapter 6 we needed exactly two instances — `BoundedJoinSemilattice Interval` (so the scheduler can merge values with $\sqcup$ and initialise cells to $\perp$) and `DecidableEq Interval` (so `Cell.write` can detect whether the value actually changed) — and both are now in place. What remains is the arithmetic itself, which must handle unbounded ends: a finite number plus “anything at all” is again “anything at all”.

```

1 namespace Intervals
2
3 -- Adding bounds: if either end is unbounded, so is the sum.
4 def addBound : Bound → Bound → Bound
5   | some a, some b => some (a + b)
6   | _,       _      => none
7
8 def subBound : Bound → Bound → Bound
9   | some a, some b => some (a - b)

```

```

10 | _, _ => none
11
12 theorem ordered_addBound {l1 h1 l2 h2 : Bound} :
13   ordered l1 h1 → ordered l2 h2 →
14   ordered (addBound l1 l2) (addBound h1 h2) := by
15   cases l1 <=> cases l2 <=> cases h1 <=> cases h2 <=>
16   simp [ordered, addBound] <=> omega
17
18 theorem ordered_subBound {l1 h1 l2 h2 : Bound} :
19   ordered l1 h1 → ordered l2 h2 →
20   ordered (subBound l1 h2) (subBound h1 l2) := by
21   cases l1 <=> cases l2 <=> cases h1 <=> cases h2 <=>
22   simp [ordered, subBound] <=> omega
23
24 -- [l1,h1] + [l2,h2] = [l1+l2, h1+h2]
25 def Interval.add : Interval → Interval → Interval
26 | .empty, _ | _, .empty => .empty
27 | .range l1 h1 ok1, .range l2 h2 ok2 =>
28   .range (addBound l1 l2) (addBound h1 h2) (ordered_addBound ok1 ok2)
29
30 -- [l1,h1] - [l2,h2] = [l1-h2, h1-l2]
31 def Interval.sub : Interval → Interval → Interval
32 | .empty, _ | _, .empty => .empty
33 | .range l1 h1 ok1, .range l2 h2 ok2 =>
34   .range (subBound l1 h2) (subBound h1 l2) (ordered_subBound ok1 ok2)
35
36 end Intervals

```

Note the asymmetry in subtraction: the smallest possible value of $x - y$ pairs the smallest x with the *largest* y , hence $[l_1 - h_2, h_1 - l_2]$. The proof obligations on the results are discharged by the two `ordered\_*` lemmas, whose hypotheses are exactly the `ok` fields of the argument intervals.

7.5 A Worked Example: Bound Propagation

Example — $x + y = 10, x \in [2, 8]$

What is the tightest interval for y ?

From $x + y = 10$ we get $y = 10 - x$. Since $x \in [2, 8]$, we have $y \in [10 - 8, 10 - 2] = [2, 8]$. The propagator should derive this automatically.

```

1 namespace Intervals
2
3 -- Propagator: sum = x + y (bidirectional: also narrows x and y)
4 def addConstraint

```



```

5   (cx cy csum : Propagators.Cell Interval) : Array Propagators.PropStep :=
6   #[
7     -- Forward: sum ← x + y
8     do
9       let x ← cx.read; let y ← cy.read
10      csum.write (Interval.add x y),
11      -- Backward: x ← sum - y
12      do
13        let s ← csum.read; let y ← cy.read
14        cx.write (Interval.sub s y),
15        -- Backward: y ← sum - x
16        do
17          let s ← csum.read; let x ← cx.read
18          cy.write (Interval.sub s x)
19    ]
20
21  def showLo : Bound → String
22  | none   => "-∞"
23  | some n => toString n
24
25  def showHi : Bound → String
26  | none   => "+∞"
27  | some n => toString n
28
29  def Interval.show : Interval → String
30  | .empty => "∅ (contradiction)"
31  | .range l h _ => s!["[{showLo l}, {showHi h}]"
32
33  def intervalExample : IO Unit := do
34    let cx ← Propagators.Cell.new (α := Interval)
35    let cy ← Propagators.Cell.new (α := Interval)
36    let csum ← Propagators.Cell.new (α := Interval)
37
38    -- x ∈ [2, 8] and x + y = 10 (i.e. sum ∈ [10, 10])
39    let _ ← cx.write (Interval.between 2 8)
40    let _ ← csum.write (Interval.between 10 10)
41
42    Propagators.runToFixpoint (addConstraint cx cy csum)
43
44    IO.println s!"x   ∈ {Interval.show (← cx.read)}"
45    IO.println s!"y   ∈ {Interval.show (← cy.read)}"
46    IO.println s!"sum ∈ {Interval.show (← csum.read)}"
47
48  #eval intervalExample
49
50  end Intervals

```

Running it, the network — forward *and* backward propagators installed — reaches quiescence and prints:

```
x   ∈ [2, 8]
y   ∈ [2, 8]
sum ∈ [10, 10]
```

The join in action. Pass 1: the forward propagator reads $x = [2, 8]$ and $y = \perp$, computes $x + y = (-\infty, +\infty)$, and merges it into c_{sum} — no change, since $[10, 10] \sqcup \perp = [10, 10]$. The backward propagator for y computes $\text{sum} - x = [10 - 8, 10 - 2] = [2, 8]$ and merges it into c_y : $\perp \sqcup [2, 8] = [2, 8]$, so c_y narrows. Pass 2: the forward propagator now computes $x + y = [4, 16]$ and merges: $[10, 10] \sqcup [4, 16] = [\max(10, 4), \min(10, 16)] = [10, 10]$. Nothing widens and nothing is lost — the join is intersection, so a looser derived constraint disappears into a tighter stored one. The pass changes nothing, the queue drains, and the network is quiescent. (With a hull-shaped “widening” join instead, that same forward write would smear $[10, 10]$ out to $[4, 16]$, the backward propagators would widen x and y in response, and the network would never quiesce.)

Termination, honestly. Chapter 5’s convergence theorem assumes the ascending chain condition, and the interval lattice does *not* satisfy it: $(-\infty, +\infty) < [0, +\infty) < [1, +\infty) < \dots$ climbs forever. Quiescence above is therefore a property of *this constraint system*, not of the lattice: these propagators happen to hit their least fixed point after two passes. A system like $y = x + 1$, $x = y + 1$ seeded with $x \in [0, +\infty)$ really would tighten forever; industrial interval solvers add a *widening* operator that jumps a bound straight to $\pm\infty$ after a few rounds to force termination. The ACC is sufficient for termination, never necessary.

7.6 Exercises

Exercise 7.1 — The hull is the meet

The join of the interval lattice is intersection. Show that the lattice also has all *meets*: define `Interval.inf`, the *hull* (smallest interval containing both arguments — the looser bound on each side, with `Interval.inf .empty x = x`), and prove the analogue of `inf_le_left`. Why does the hull, which *loses* information, sit *below* its arguments in the information order?

Exercise 7.2 — Multiplication intervals

Implement `Interval.mul`. Be careful on two fronts: multiplying finite intervals with negative ends requires considering all four corner products, and unbounded ends can flip sign — e.g. $[-2, -1] \times [0, +\infty) = (-\infty, 0]$ — so `mulBound` needs genuine

sign analysis rather than a one-line match.

Part III

Capstone Projects

Chapter 8

Capstone I: A Sudoku Solver

8.1 What is Sudoku?

Sudoku is a logic puzzle played on a 9×9 grid. The grid is divided into nine 3×3 *boxes*. Some cells are pre-filled with digits; your task is to fill in the rest.

r1	5	3	4	6	7	8	9	1	2
r2	6	box top-left 2	1	9	5	3	4	8	
r3	1	9	8	3	4	2	5	6	7
r4	8				6				3
r5	4			8		3			1
r6	7				2				6
r7		6					2	8	
r8				4	1	9			5
r9					8			7	9
	c1	c2	c3	c4	c5	c6	c7	c8	c9

Figure 8.1: A Sudoku puzzle. Bold digits are givens; light grey digits show a few solved cells. The thick lines divide the grid into nine 3×3 boxes.

The three rules. Every digit 1–9 must appear *exactly once* in each of:

- each **row** (9 rows, one per horizontal strip),
- each **column** (9 columns, one per vertical strip),
- each **box** (9 boxes, each a 3×3 sub-grid).

That is the entire rule set. No arithmetic is involved — the digits could equally well be colours, letters, or symbols. The puzzle is purely about placement constraints.

How to solve by hand. The standard technique is *elimination*: for each empty cell, maintain the set of digits that could still legally go there given the filled cells in the same row, column, and box. If a cell's candidate set shrinks to a single digit, that digit is determined. Filling it in eliminates that digit from the candidate sets of neighbouring cells, which may in turn determine more cells. This cascading process is exactly what a propagator network formalises.

Example — A simple deduction

Suppose row 1 already contains 1, 2, 3, 4, 6, 7, 8, 9 in various cells. The only missing digit is 5, so the one remaining empty cell in row 1 must be 5 — no other placement is consistent with the row constraint.

Example — Elimination across constraints

Suppose a cell sits in row 3, column 5, box 5 (the centre box). Row 3 already contains $\{1, 3, 4, 6, 7\}$, column 5 contains $\{2, 3, 6, 8\}$, and box 5 contains $\{1, 2, 6, 9\}$. The candidates for this cell are all digits *not* ruled out by any of its three units:

$$\{1, \dots, 9\} \setminus (\{1, 3, 4, 6, 7\} \cup \{2, 3, 6, 8\} \cup \{1, 2, 6, 9\}) = \{5\}.$$

One candidate remains, so this cell must be 5. Each elimination step is an intersection with the complement of a forbidden set — computed with `CandidateSet.inter`, which is exactly the join of the information order.

Why propagator networks are a natural fit. Each constraint (row, column, box) is a propagator that reads the determined cells in its unit and writes the resulting eliminations to undetermined cells. The network runs until quiescence — no propagator can make further eliminations. This is guaranteed to terminate because candidate sets can only shrink, and a finite set can only shrink finitely many times. Whether it *solves* the puzzle completely depends on the puzzle; harder puzzles require search in addition to propagation.

8.2 The Domain Lattice

Sudoku is a constraint satisfaction problem: each of 81 cells must receive a digit 1–9 subject to row, column, and box uniqueness constraints. Propagator networks handle this beautifully. We build a complete solver using nothing but lattice-ordered propagation.

Each Sudoku cell can be modelled as a set of *candidate digits*. Initially every cell can hold any of $\{1, \dots, 9\}$. Constraints eliminate candidates.

Definition — Candidate Set Lattice

Let $D = \{1, \dots, 9\}$. The *candidate-set lattice* is the powerset $\mathcal{P}(D)$ ordered by *reverse inclusion*:

$$S \leq T \iff T \subseteq S.$$

The bottom $\perp = D$ (no information: any digit possible), the top $\top = \emptyset$ (contradiction: no digit possible). The join is intersection: $S \sqcup T = S \cap T$ (the smallest set of candidates consistent with both sources).

This is precisely the *dual* powerset lattice from Exercise 1.4!

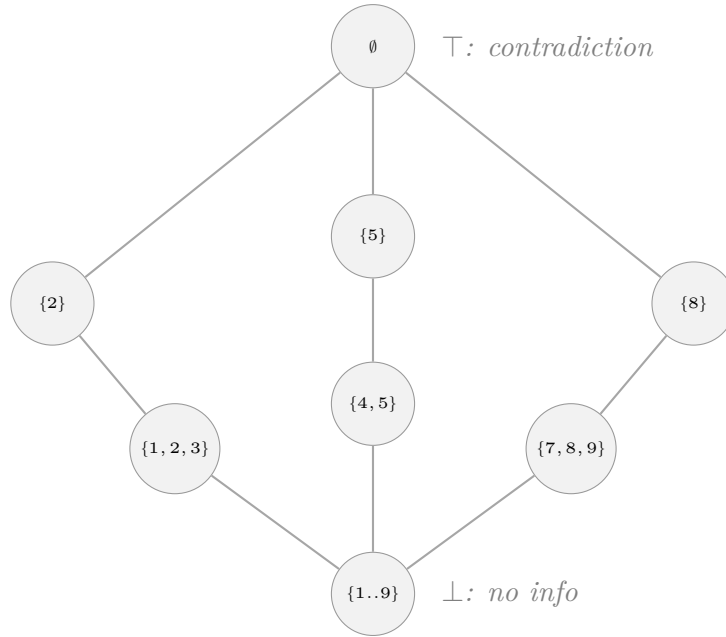


Figure 8.2: A fragment of the candidate-set lattice (reverse inclusion order).

```

1 namespace Sudoku
2
3 -- Digits 1..9, represented as Fin 9 (0..8 internally, displayed as 1..9)
4 abbrev Digit := Fin 9
5
6 -- A candidate set: which digits are still possible for this cell
7 -- We represent it as a Bool array of length 9
8 structure CandidateSet where
9   present : Fin 9 → Bool
10
11 def CandidateSet.full : CandidateSet :=
12   { present := fun _ => true }
13
14 def CandidateSet.empty : CandidateSet :=
15   { present := fun _ => false }
16
17 def CandidateSet.singleton (d : Digit) : CandidateSet :=
18   { present := fun i => i == d }
19
20 def CandidateSet.remove (s : CandidateSet) (d : Digit) : CandidateSet :=
21   { present := fun i => s.present i && !(i == d) }
22
23 def CandidateSet.inter (s t : CandidateSet) : CandidateSet :=
24   { present := fun i => s.present i && t.present i }
25
26 def CandidateSet.union (s t : CandidateSet) : CandidateSet :=

```

```

27 { present := fun i => s.present i || t.present i }
28
29 -- The information order:  $s \leq t$  iff  $t \subseteq s$  ( $t$  is more specific)
30 def CandidateSet.le (s t : CandidateSet) : Prop :=
31    $\forall i : \text{Digit}, t.\text{present } i = \text{true} \rightarrow s.\text{present } i = \text{true}$ 
32
33 -- Antisymmetry: compare the two indicator functions pointwise
34 -- (rename_i names the functions exposed by `cases`).
35 instance : OrderBook.PartialOrder CandidateSet where
36   le      := CandidateSet.le
37   le_refl := fun s i h => h
38   le_antisymm := fun s t h1 h2 =>
39     show s = t from by
40       cases s; cases t
41       congr 1; funext i
42       rename_i sp tp
43       cases hi : tp i
44       · cases hs : sp i
45         · rfl
46         · exact absurd (h2 i hs) (by simp [hi])
47       · exact h1 i hi
48   le_trans := fun s t u hst htu i hi => hst i (htu i hi)
49
50 -- Join = intersection (combines constraints).
51 -- `Bool.and_eq_true : ((a && b) = true) = (a = true  $\wedge$  b = true)` is
52 -- an equality of propositions, so we rewrite with `>`.
53 instance : Propagators.BoundedJoinSemilattice CandidateSet where
54   le      := CandidateSet.le
55   le_refl := fun s i h => h
56   le_antisymm := OrderBook.PartialOrder.le_antisymm ( $\alpha$  := CandidateSet)
57   le_trans := OrderBook.PartialOrder.le_trans ( $\alpha$  := CandidateSet)
58   bot      := CandidateSet.full
59   sup      := CandidateSet.inter --  $U = n$  in information order!
60   bot_le   := fun s i _ => rfl --  $\text{full} \leq s$  since  $s \subseteq \text{full}$ 
61   le_sup_left := fun s t i h => (Bool.and_eq_true _ _ > h).1
62   le_sup_right := fun s t i h => (Bool.and_eq_true _ _ > h).2
63   sup_le     := fun s t u hsu htu i h =>
64     Bool.and_eq_true _ _ > ((hsu i h, htu i h) : _  $\wedge$  _)
65
66 instance : DecidableEq CandidateSet :=
67   fun s t =>
68     if h :  $\forall i, s.\text{present } i = t.\text{present } i$ 
69     then isTrue (by cases s; cases t; congr 1; funext i; exact h i)
70     else isFalse (fun heq => h (fun i => by rw [heq]))
71
72 end Sudoku

```

8.3 Representing the Grid

```

1 namespace Sudoku
2
3 -- 81 cells indexed by row and column (both Fin 9)
4 abbrev Grid := Fin 9 → Fin 9 → Propagators.Cell CandidateSet
5
6 -- The `[·]!` index operators need an `Inhabited` fallback value;
7 -- `IO.Ref` provides none, so we allocate one dummy cell and bring a
8 -- local `Inhabited` instance into scope.
9 def makeGrid : IO Grid := do
10   let mut rows : Array (Array (Propagators.Cell CandidateSet)) := #[]
11   for _ in [:9] do
12     let mut row : Array (Propagators.Cell CandidateSet) := #[]
13     for _ in [:9] do
14       let c ← Propagators.Cell.new (α := CandidateSet)
15       row := row.push c
16     rows := rows.push row
17   let dummy ← Propagators.Cell.new (α := CandidateSet)
18   have : Inhabited (Propagators.Cell CandidateSet) := ⟨dummy⟩
19   return fun r c => rows[r.val]![c.val]!
20
21 -- Seed a given digit into a cell
22 def seedCell (grid : Grid) (r c : Fin 9) (d : Digit) : IO Unit := do
23   let _ ← (grid r c).write (CandidateSet.singleton d)
24
25 end Sudoku

```

8.4 Constraint Propagators

Sudoku has three families of constraints: row, column, and 3×3 box uniqueness. Each is expressed as a propagator that, given a cell known to hold digit d , removes d from all peers.

```

1 namespace Sudoku
2
3 -- Note the loop syntax: `for h : i in [:9]` binds, alongside i, a
4 -- proof `h` that i lies in the range; `h.upper : i < 9` is exactly
5 -- what the `Fin 9` constructor (i, _) needs.
6
7 -- If cell (r,c) is determined to be d, remove d from all other cells in row r
8 def rowPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
9   let cands ← (grid r c).read
10   -- Check if this cell is a singleton

```

```

11  let det : Option Digit := Id.run do
12    let mut found : Option Digit := none
13    let mut count := 0
14    for h : i in [:9] do
15      let i' : Digit := ⟨i, h.upper⟩
16      if cands.present i' then
17        found := some i'; count := count + 1
18    if count = 1 then found else none
19  match det with
20  | none => return false
21  | some d =>
22    let mut changed := false
23    for h : j in [:9] do
24      let j' : Fin 9 := ⟨j, h.upper⟩
25      if j' ≠ c then
26        -- We re-read and merge
27        let old ← (grid r j').read
28        let upd := CandidateSet.inter old { present := fun i => !(i == d) }
29        let c ← (grid r j').write upd
30        if c then changed := true
31    return changed
32
33  -- Analogous: column propagator
34  def colPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
35    let cands ← (grid r c).read
36    let det : Option Digit := Id.run do
37      let mut found : Option Digit := none
38      let mut count := 0
39      for h : i in [:9] do
40        let i' : Digit := ⟨i, h.upper⟩
41        if cands.present i' then found := some i'; count := count + 1
42      if count = 1 then found else none
43    match det with
44    | none => return false
45    | some d =>
46      let mut changed := false
47      for h : i in [:9] do
48        let i' : Fin 9 := ⟨i, h.upper⟩
49        if i' ≠ r then
50          let old ← (grid i' c).read
51          let upd := CandidateSet.inter old { present := fun k => !(k == d) }
52          let ch ← (grid i' c).write upd
53          if ch then changed := true
54      return changed
55
56  -- Box propagator (which 3×3 box contains (r,c)?)
57  def boxPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do

```

```

58 let cand ← (grid r c).read
59 let det : Option Digit := Id.run do
60   let mut found : Option Digit := none; let mut count := 0
61   for h : i in [:9] do
62     let i' : Digit := ⟨i, h.upper⟩
63     if cand.present i' then found := some i'; count := count + 1
64   if count = 1 then found else none
65 match det with
66 | none => return false
67 | some d =>
68   let boxR := r.val / 3 * 3
69   let boxC := c.val / 3 * 3
70   let mut changed := false
71   for hr : dr in [:3] do
72     for hc : dc in [:3] do
73       let r' : Fin 9 := ⟨boxR + dr, by have h1 : dr < 3 := hr.upper; have :=
74         ↪ r.isLt; omega⟩
75       let c' : Fin 9 := ⟨boxC + dc, by have h2 : dc < 3 := hc.upper; have :=
76         ↪ c.isLt; omega⟩
77       if !(r' = r && c' = c) then
78         let old ← (grid r' c').read
79         let upd := CandidateSet.inter old { present := fun k => !(k == d) }
80         let ch ← (grid r' c').write upd
81         if ch then changed := true
82   return changed
83 end Sudoku

```

8.5 Assembling and Running the Solver

```

1 namespace Sudoku
2
3 def buildNetwork (grid : Grid) : Array Propagators.PropStep := Id.run do
4   let mut steps : Array Propagators.PropStep := #[]
5   for hr : r in [:9] do
6     for hc : c in [:9] do
7       let r' : Fin 9 := ⟨r, hr.upper⟩
8       let c' : Fin 9 := ⟨c, hc.upper⟩
9       steps := steps.push (rowPropagator grid r' c')
10      steps := steps.push (colPropagator grid r' c')
11      steps := steps.push (boxPropagator grid r' c')
12   return steps
13
14 -- Print the grid (showing the unique digit or '?' for unsolved cells)
15 def printGrid (grid : Grid) : IO Unit := do

```

```

16 for hr : r in [:9] do
17   let mut row := ""
18   for hc : c in [:9] do
19     let r' : Fin 9 := ⟨r, hr.upper⟩
20     let c' : Fin 9 := ⟨c, hc.upper⟩
21     let cand ← (grid r' c').read
22     let mut digit := '?'
23     let mut count := 0
24     for h : i in [:9] do
25       let i' : Digit := ⟨i, h.upper⟩
26       if cand.present i' then digit := Char.ofNat (i + 49); count := count
27         ↪ + 1
28       row := row ++ (if count = 1 then s! "{digit}" else "?")
29       if c % 3 == 2 && c < 8 then row := row ++ "|"
30     IO.println row
31     if r % 3 == 2 && r < 8 then IO.println "-----"
32
33 -- Example: a simple Sudoku puzzle
34 -- (0 = unknown, digits are given clues)
35 def examplePuzzle : Array (Array Nat) := #[
36   #[5, 3, 0, 0, 7, 0, 0, 0, 0],
37   #[6, 0, 0, 1, 9, 5, 0, 0, 0],
38   #[0, 9, 8, 0, 0, 0, 0, 6, 0],
39   #[8, 0, 0, 0, 6, 0, 0, 0, 3],
40   #[4, 0, 0, 8, 0, 3, 0, 0, 1],
41   #[7, 0, 0, 0, 2, 0, 0, 0, 6],
42   #[0, 6, 0, 0, 0, 0, 2, 8, 0],
43   #[0, 0, 0, 4, 1, 9, 0, 0, 5],
44   #[0, 0, 0, 0, 8, 0, 0, 7, 9]
45 ]
46
47 -- The dependent `if h : 1 ≤ d ∧ d ≤ 9` hands omega the bound it
48 -- needs to build the `Fin 9` seed value `(d - 1, _)`.
49 def solveSudoku : IO Unit := do
50   let grid ← makeGrid
51   -- Seed known digits
52   for hr : r in [:9] do
53     for hc : c in [:9] do
54       let d := examplePuzzle[r][c]
55       if h : 1 ≤ d ∧ d ≤ 9 then
56         let r' : Fin 9 := ⟨r, hr.upper⟩
57         let c' : Fin 9 := ⟨c, hc.upper⟩
58         seedCell grid r' c' (d - 1, by omega)
59   -- Build and run the propagator network
60   let steps := buildNetwork grid
61   Propagators.runToFixpoint steps
62   -- Print the result

```

```

62  I0.println "Solution:"
63  printGrid grid
64
65  end Sudoku

```

Running `##eval solveSudoku`, propagation alone drives every cell to a singleton — this puzzle needs no search — and prints the grid’s unique solution:

```

Solution:
534|678|912
672|195|348
198|342|567
-----
859|761|423
426|853|791
713|924|856
-----
961|537|284
287|419|635
345|286|179

```

8.6 Why This Works: The Lattice Perspective

- Each cell starts at $\perp = \{1, \dots, 9\}$: no information.
- Every propagator only writes using `CandidateSet.inter` (the join in the information order), which can only *reduce* candidate sets — i.e., move *up* in the information order.
- The network terminates because the candidate-set lattice is finite and the cells only move upward.
- Reaching $\top = \emptyset$ in any cell signals a contradiction.
- A solved cell is one that has reached a singleton $\{d\}$.

The elegant insight: *solving Sudoku is computing the least fixed point of the combined propagation function*, where the partial order is the information order on candidate sets. Knaster–Tarski guarantees it exists; our scheduler computes it.

Exercise 8.1 — Naked pair

A *naked pair* occurs when two cells in the same unit both have exactly the same two candidates $\{a, b\}$. Then a and b can be removed from all other cells in that unit. Implement this as an additional propagator.

Exercise 8.2 — Contradiction detection

Add a `contradictionProp` that checks every cell; if any cell reaches $\top = \emptyset$, print a diagnostic and halt. Why is this detection sound (i.e., why can't a cell reach $\emptyset$ unless the puzzle is truly unsatisfiable)?

Chapter 9

Capstone II: Type Inference via Propagation

Our second capstone takes us into programming language theory. Type inference — the process of automatically deducing types of expressions — is naturally formulated as constraint propagation on a type lattice.

9.1 A Simple Expression Language

We work with a fragment of the lambda calculus extended with integers and booleans.

```
1 namespace TypeInfer
2
3 -- Expressions
4 inductive Expr where
5   | intLit (n : Int)      : Expr
6   | boolLit (b : Bool)   : Expr
7   | var     (x : String)  : Expr
8   | add     (e1 e2 : Expr) : Expr
9   | ifExpr  (cond t f : Expr) : Expr
10  | lam     (x : String) (body : Expr) : Expr
11  | app     (fn arg : Expr) : Expr
12  | letExpr (x : String) (e body : Expr) : Expr
13
14 end TypeInfer
```

9.2 The Type Lattice

Rather than types being a simple set, we arrange them in a lattice that captures the degree to which a type is *known*.

Definition — Flat Type Lattice

Define the type information domain:

- $\perp$ (Unknown): no type information yet.
- TInt , TBool : concrete base types.
- $\text{TFun } s \ t$: function type, parameterised by the type information for the domain and codomain (which are themselves in the lattice).
- $\top$ (TypeError): contradiction (incompatible constraints).

The order is: $\perp \leq$ everything, everything $\leq \top$, and base types are only $\leq$ themselves (and $\perp/\top$). Function types: $\text{TFun } s_1 \ t_1 \leq \text{TFun } s_2 \ t_2$ iff $s_1 \leq s_2 \wedge t_1 \leq t_2$.

```

1 namespace TypeInfer
2
3 inductive Ty where
4   | unknown      : Ty    --  $\perp$ 
5   | int          : Ty
6   | bool         : Ty
7   | fun_ (dom cod : Ty) : Ty    --  $\text{dom} \rightarrow \text{cod}$ 
8   | typeError    : Ty    --  $\top$ 
9
10 -- Unification of two types: compute their join
11 def Ty.unify : Ty → Ty → Ty
12   | .unknown, t      => t
13   | t, .unknown      => t
14   | .typeError, _    => .typeError
15   | _, .typeError    => .typeError
16   | .int, .int        => .int
17   | .bool, .bool      => .bool
18   | .fun_ d1 c1, .fun_ d2 c2 => .fun_ (Ty.unify d1 d2) (Ty.unify c1 c2)
19   | _, _              => .typeError -- incompatible types
20
21 -- The information order
22 def Ty.le : Ty → Ty → Prop
23   | .unknown, _      => True
24   | _, .typeError    => True
25   | .int, .int        => True
26   | .bool, .bool      => True
27   | .fun_ d1 c1, .fun_ d2 c2 => Ty.le d1 d2 ∧ Ty.le c1 c2
28   | _, _              => False
29
30 -- The order laws are proved as standalone lemmas by structural
31 -- induction on the first argument (the recursive `fun_` constructor
32 -- needs genuine induction hypotheses), then plugged into the
33 -- instances as terms.
34

```

```

35 theorem Ty.le_refl_thm :  $\forall$  (t : Ty), Ty.le t t := by
36   intro t
37   induction t <;> simp [Ty.le, *]
38
39 theorem Ty.le_antisymm_thm :  $\forall$  (a b : Ty), Ty.le a b  $\rightarrow$  Ty.le b a  $\rightarrow$  a = b := by
40   intro a
41   induction a with
42   | fun_ d c ihd ihc =>
43     intro b h1 h2
44     cases b <;> simp_all [Ty.le]
45     exact (ihd _ h1.1 h2.1, ihc _ h1.2 h2.2)
46   | _ => intro b h1 h2; cases b <;> simp_all [Ty.le]
47
48 theorem Ty.le_trans_thm :  $\forall$  (a b c : Ty), Ty.le a b  $\rightarrow$  Ty.le b c  $\rightarrow$  Ty.le a c :=
49    $\hookrightarrow$  by
50   intro a
51   induction a with
52   | fun_ d c ihd ihc =>
53     intro b c' h1 h2
54     cases b <;> cases c' <;> simp_all [Ty.le]
55     exact (ihd _ _ h1.1 h2.1, ihc _ _ h1.2 h2.2)
56   | _ => intro b c' h1 h2; cases b <;> cases c' <;> simp_all [Ty.le]
57
58 theorem Ty.le_sup_left_thm :  $\forall$  (a b : Ty), Ty.le a (Ty.unify a b) := by
59   intro a
60   induction a with
61   | fun_ d c ihd ihc =>
62     intro b
63     cases b <;> simp [Ty.le, Ty.unify]
64     · exact (Ty.le_refl_thm d, Ty.le_refl_thm c)
65     · exact (ihd _, ihc _)
66   | _ => intro b; cases b <;> simp [Ty.le, Ty.unify]
67
68 theorem Ty.le_sup_right_thm :  $\forall$  (a b : Ty), Ty.le b (Ty.unify a b) := by
69   intro a
70   induction a with
71   | fun_ d c ihd ihc =>
72     intro b
73     cases b <;> simp [Ty.le, Ty.unify]
74     · exact (ihd _, ihc _)
75   | _ => intro b; cases b <;> simp [Ty.le, Ty.unify, Ty.le_refl_thm]
76
77 theorem Ty.sup_le_thm :
78    $\forall$  (a b c : Ty), Ty.le a c  $\rightarrow$  Ty.le b c  $\rightarrow$  Ty.le (Ty.unify a b) c := by
79   intro a
80   induction a with
81   | fun_ d c ihd ihc =>

```

```

81   intro b c' h1 h2
82   cases b <=> cases c' <=> simp_all [Ty.le, Ty.unify]
83 | _ => intro b c' h1 h2; cases b <=> cases c' <=> simp_all [Ty.le, Ty.unify]
84
85 instance : OrderBook.PartialOrder Ty where
86   le      := Ty.le
87   le_refl := Ty.le_refl_thm
88   le_antisymm := Ty.le_antisymm_thm
89   le_trans := Ty.le_trans_thm
90
91 instance : Propagators.BoundedJoinSemilattice Ty where
92   le      := Ty.le
93   le_refl := Ty.le_refl_thm
94   le_antisymm := Ty.le_antisymm_thm
95   le_trans := Ty.le_trans_thm
96   bot      := .unknown
97   sup      := Ty.unify
98   bot_le   := fun _ => trivial
99   le_sup_left  := Ty.le_sup_left_thm
100  le_sup_right := Ty.le_sup_right_thm
101  sup_le      := Ty.sup_le_thm
102
103 deriving instance DecidableEq for Ty
104
105 end TypeInfer

```

Why lemmas instead of inline tactic blocks? `Ty` is our first *recursive* lattice: `fun \_ dom cod` contains two smaller `Tys`, so every law needs genuine induction hypotheses about those components, and the tidiest way to manage them is `induction a with` naming the hypotheses (`ihd`, `ihc`) explicitly. The wildcard arm `| \_ =>` covers the four non-recursive constructors at once.

9.3 Type Constraint Generation

We generate constraints by walking the expression tree. Each sub-expression gets a fresh *type cell*. Constraints are propagators that merge type information into these cells. (A syntactic trap to avoid: the propagators are appended with `steps.modify (·.push ...)`, and the anonymous-function dot must *not* be followed by a space — `(·.push ...)` parses as applying the placeholder to `.push` and fails with “function expected”.)

```

1 namespace TypeInfer
2
3 -- A typing context maps variable names to type cells
4 abbrev Context := List (String × Propagators.Cell Ty)
5

```

```

6  def Context.lookup (ctx : Context) (x : String) :
7    Option (Propagators.Cell Ty) :=
8    ctx.find? (fun p => p.1 == x) |>.map (·.2)
9
10  -- Generate type cells and propagators for an expression
11  -- Returns the type cell for this expression.
12  partial def inferExpr
13    (ctx : Context)
14    (e : Expr)
15    (steps : IO.Ref (Array Propagators.PropStep)) :
16    IO (Propagators.Cell Ty) := do
17    match e with
18    | .intLit _ =>
19      let c ← Propagators.Cell.new (α := Ty)
20      let _ ← c.write .int
21      return c
22
23    | .boolLit _ =>
24      let c ← Propagators.Cell.new (α := Ty)
25      let _ ← c.write .bool
26      return c
27
28    | .var x =>
29      match ctx.lookup x with
30      | some c => return c
31      | none   =>
32        let c ← Propagators.Cell.new (α := Ty)
33        let _ ← c.write .typeError -- unbound variable
34        return c
35
36    | .add e1 e2 =>
37      let c1 ← inferExpr ctx e1 steps
38      let c2 ← inferExpr ctx e2 steps
39      let cOut ← Propagators.Cell.new (α := Ty)
40      -- Propagator: both operands must be Int. We then re-read the
41      -- operand cells and merge their join into the output – so if an
42      -- operand has jumped to typeError (e.g. Bool ⊔ Int), the error
43      -- reaches the result cell too.
44      steps.modify (·.push do
45        let _ ← c1.write .int
46        let _ ← c2.write .int
47        let t1 ← c1.read; let t2 ← c2.read
48        cOut.write (Ty.unify t1 t2))
49      return cOut
50
51    | .ifExpr cond t f =>
52      let ccond ← inferExpr ctx cond steps

```

```

53   let ct    ← inferExpr ctx t    steps
54   let cf    ← inferExpr ctx f    steps
55   let cOut  ← Propagators.Cell.new (α := Ty)
56   -- Propagator: cond must be Bool; t and f must agree; out = t
57   steps.modify (·.push do
58     let _ ← ccond.write .bool
59     let tv ← ct.read; let fv ← cf.read
60     let _ ← ct.write fv; let _ ← cf.write tv
61     let tv' ← ct.read
62     cOut.write tv')
63   return cOut
64
65 | .lam x body =>
66   let cDom  ← Propagators.Cell.new (α := Ty)
67   let ctx'  := (x, cDom) :: ctx
68   let cBody ← inferExpr ctx' body steps
69   let cOut  ← Propagators.Cell.new (α := Ty)
70   -- Propagator: out = dom → body
71   steps.modify (·.push do
72     let d ← cDom.read; let b ← cBody.read
73     cOut.write (.fun_ d b))
74   return cOut
75
76 | .app fn arg =>
77   let cFn   ← inferExpr ctx fn  steps
78   let cArg  ← inferExpr ctx arg steps
79   let cOut  ← Propagators.Cell.new (α := Ty)
80   -- Propagator: fn must be (arg → out)
81   steps.modify (·.push do
82     let fv ← cFn.read; let av ← cArg.read
83     let ov ← cOut.read
84     match fv with
85     | .fun_ d r =>
86       let _ ← cArg.write d    -- arg type must match domain
87       let _ ← cOut.write r    -- result type from codomain
88       let _ ← cFn.write (.fun_ av ov) -- refine fn type
89       return false           -- (actual change tracked by writes)
90     | .unknown =>
91       let _ ← cFn.write (.fun_ av ov)
92       return false
93     | _ =>
94       let _ ← cFn.write .typeError; return false)
95   return cOut
96
97 | .letExpr x e body =>
98   let ce   ← inferExpr ctx e    steps
99   let ctx' := (x, ce) :: ctx

```

```

100   inferExpr ctx' body steps
101
102 end TypeInfer

```

9.4 Running the Type Inferencer

```

1 namespace TypeInfer
2
3 def infer (e : Expr) : IO Ty := do
4   let stepsRef ← IO.mkRef (Array.empty : Array Propagators.PropStep)
5   let cResult  ← inferExpr [] e stepsRef
6   let steps    ← stepsRef.get
7   Propagators.runToFixpoint steps
8   cResult.read
9
10  -- Helper to show types
11  def Ty.toString : Ty → String
12    | .unknown      => "?"
13    | .int          => "Int"
14    | .bool         => "Bool"
15    | .fun_ d r     => s! "{Ty.toString d} → {Ty.toString r}"
16    | .typeError    => "TypeError"
17
18  -- Examples
19  def ex1 : Expr := .add (.intLit 1) (.intLit 2)
20  -- Expected: Int
21
22  def ex2 : Expr := .lam "x" (.add (.var "x") (.intLit 1))
23  -- Expected: (Int → Int)
24
25  def ex3 : Expr := .ifExpr (.boolLit true) (.intLit 1) (.intLit 2)
26  -- Expected: Int
27
28  def ex4 : Expr := .add (.boolLit true) (.intLit 1)
29  -- Expected: TypeError (Bool ≠ Int)
30
31  def runExamples : IO Unit := do
32    for (name, e) in [("1 + 2", ex1), ("λx. x+1", ex2),
33                     ("if true 1 2", ex3), ("true + 1", ex4)] do
34      let ty ← infer e
35      IO.println s! "{name} : {Ty.toString ty}"
36
37 end TypeInfer

```

Expected output:

```

1 + 2 : Int
λx. x+1 : (Int → Int)
if true 1 2 : Int
true + 1 : TypeError

```

9.5 What the Lattice Buys Us Here

- **Bidirectional propagation is free.** When we call `cArg.write d` inside the `app` propagator, we are propagating type information *backwards*: from the function type to the argument. The lattice join ensures this never loses previously-inferred information.
- **Inconsistency is explicit.** `TypeError` is the $\top$ element. When two incompatible type constraints are merged (`Ty.unify .int .bool = .typeError`), the cell value jumps to $\top$ and stays there. No exception is thrown; the type error propagates naturally through the lattice.
- **Incrementality is free.** If we learn a new fact about x 's type (say, x appears in another expression), we just write to x 's type cell and re-run the network. The lattice join handles merging old and new constraints correctly.
- **Ordering captures “how much we know”.** $\perp = \text{Unknown} < \text{Int} < \text{TypeError} = \top$. The height of a cell's value in the lattice is a measure of how constrained it is. A fully-typed program has all cells at concrete types (neither $\perp$ nor $\top$).

9.6 Exercises

Exercise 9.1 — Let-polymorphism

Lean's type system supports polymorphism. Extend `Ty` with a `TVar (name : String)` constructor and modify `Ty.unify` to do proper unification (substituting type variables). How does this affect the lattice structure?

Exercise 9.2 — Recursive types

Add a `letRec` construct to `Expr` and extend the inferencer to handle it. Note: the type cell for the recursive binding must be initialised to `Unknown` and refined by propagation, enabling inference of e.g. `let rec f x = if x = 0 then 1 else x * f (x-1).`

Exercise 9.3 — Error messages

When a cell reaches `TypeError`, record *why* (which two types conflicted). Augment the `Ty` type so that `TypeError` carries a `String` explanation, and thread error mes-

sages through `Ty.unify`.

Appendix A

Lean 4 Quick Reference and Tactic Primer

A.1 Basic Types and Syntax

```
1 -- Type universes
2 #check Nat      -- Nat : Type
3 #check Type     -- Type : Type 1
4 #check Prop     -- Prop : Type
5
6 -- Function types
7 def f : Nat → Nat → Nat := fun a b => a + b
8 def g (a b : Nat) : Nat := a + b    -- same, different syntax
9
10 -- Dependent types (e.g. fixed-length vectors as functions)
11 def myList (α : Type) (n : Nat) : Type := Fin n → α
12
13 -- Let bindings
14 def h : Nat :=
15   let x := 5
16   let y := x + 1
17   x * y
```

A.2 Typeclasses

```
1 -- Declaring a typeclass
2 class MyClass (α : Type) where
3   someMethod : α → Nat
4   someProperty : ∀ (a : α), someMethod a ≥ 0
5
6 -- Providing an instance
7 instance : MyClass Nat where
8   someMethod := id
9   someProperty := Nat.zero_le
```

```
10
11 -- Using an instance
12 def useMyClass [MyClass  $\alpha$ ] (a :  $\alpha$ ) : Nat :=
13   MyClass.someMethod a
```

A.3 Tactic Primer

A theorem in Lean is a type; a proof is a term of that type. You can write that term directly, or enter *tactic mode* with **by** and build it interactively: at every point there is a *goal* (hypotheses above the line, target below), and each tactic transforms the goal until none remain. Tactics are not magic — each one *elaborates to an ordinary term* in the end, and for the most important ones this primer shows both the tactic and the term it stands for. Every example below is compile-checked against the same zero-dependency toolchain as the rest of the book.

intro and exact

intro *h* moves the premise of an implication (or the bound variable of a $\forall$) into the hypotheses; **exact** *e* closes the goal with the term *e*, which must fit exactly.

```
1 theorem imp_demo (p q : Prop) (hq : q) : p → q ∧ q := by
2   intro hp
3   exact ⟨hq, hq⟩
```

The angle brackets `⟨...⟩` build a value of any one-constructor type (here `And.intro`); this proof is literally the term `fun hp => ⟨hq, hq⟩`.

apply

apply *f* matches the goal against the *conclusion* of *f* and turns *f*'s remaining arguments into new goals — backward reasoning.

```
1 theorem apply_demo (a b : Nat) (h : a = b) : a ≤ b := by
2   apply Nat.le_of_eq -- goal becomes: a = b
3   exact h
```

rfl

Closes goals of the form `a = a` — *up to computation*. Lean evaluates both sides, so `rfl` proves more than it looks like it should:

```
1 example : 2 + 2 = 4 := rfl
2 example : (fun n : Nat => n + 1) 3 = 4 := rfl
```

Because proofs of the same proposition are definitionally equal (*proof irrelevance*), `rfl` also closes equalities between structures that differ only in proof fields — Chapter 7’s interval antisymmetry ends this way.

constructor

Applies the (unique) constructor of the goal’s type: an `And` goal splits into two goals, an `Iff` into two implications, and so on. The bullets `·` focus one subgoal each.

```
1 example : 1 ≤ 2 ∧ 2 ≤ 3 := by
2   constructor
3   · omega
4   · omega
```

cases — and what it desugars to

`cases x` splits the proof into one branch per constructor of `x`’s inductive type. This is not a primitive: every inductive type `T` comes with a *recursor* `T.rec`, a function that says “to define something for all `T`, handle each constructor”. `cases` (and `induction`) are sugar for applying it.

```
1 theorem bool_or_not (b : Bool) : (b || !b) = true := by
2   cases b with
3   | false => rfl
4   | true  => rfl
5
6 -- the same proof written directly against the recursor
7 theorem bool_or_not' (b : Bool) : (b || !b) = true :=
8   Bool.rec (motive := fun x => (x || !x) = true) rfl rfl b
```

The `motive` is the family of statements being proved — “what does the goal look like as a function of `x`?” — and the two `rfl`s are the `false` and `true` branches.

induction — and what it desugars to

Same recursor, but for recursive types the branch for a recursive constructor receives an *induction hypothesis*. Note how precisely the term mirrors the tactic script:

```
1 theorem zero_add_tac (n : Nat) : 0 + n = n := by
2   induction n with
3   | zero => rfl
4   | succ k ih => rw [Nat.add_succ, ih]
5
6 -- the same proof as an explicit Nat.rec term
7 theorem zero_add_term (n : Nat) : 0 + n = n :=
```

```

8   Nat.rec (motive := fun m => 0 + m = m)
9     rfl
10    (fun k ih => congrArg Nat.succ ih)
11    n

```

`Nat.rec` takes the base case (a proof for `0`), the step case (a function from `k` and the induction hypothesis `ih` to a proof for `k + 1`), and the number to recurse on. *Induction is just recursion over proofs.*

`obtain` / `rcases` — and what they desugar to

Both destructure a hypothesis by pattern: `obtain (k, hk) := h` takes `h : ∃ k, P k` apart into a witness `k` and a proof `hk : P k`. (`rcases h with (k, hk)` is the same with recursive patterns like `(a, (b, c))`.) Underneath it is the eliminator of `Exists` — or, equivalently, a `match`:

```

1  theorem pos_of_succ {n : Nat} (h : ∃ k, n = k + 1) : 0 < n := by
2    obtain (k, hk) := h
3    subst hk
4    exact Nat.succ_pos k
5
6  -- as an application of the eliminator ...
7  theorem pos_of_succ' {n : Nat} (h : ∃ k, n = k + 1) : 0 < n :=
8    Exists.elim h (fun k hk => hk ▸ Nat.succ_pos k)
9
10  -- ... or as a match
11  theorem pos_of_succ'' {n : Nat} (h : ∃ k, n = k + 1) : 0 < n :=
12    match h with
13    | (k, hk) => hk ▸ Nat.succ_pos k

```

(`h ▸ e` rewrites the goal with the equation `h` and then supplies `e`; it is the term-level cousin of `rw`.)

`rename_i`

Tactics like `cases` sometimes introduce hypotheses with inaccessible machine-generated names (shown with a dagger, e.g. `a†`). `rename_i x` gives the most recent such hypothesis the name `x` so you can refer to it.

```

1  theorem rename_demo (o : Option Nat) (h : o ≠ none) : ∃ n, o = some n := by
2    cases o
3    · exact absurd rfl h
4    · rename_i n      -- name the anonymous value introduced by `cases`
5    exact (n, rfl)

```

by_cases — and the Decidable if underneath

by_cases $h : p$ splits the proof into a branch where $h : p$ and one where $h : \neg p$. In this book's zero-dependency setting it needs p to be *decidable* — there must be an algorithm answering p — because it is sugar for a dependent **if**:

```

1 theorem em_nat (n : Nat) : n = 0 ∨ n ≠ 0 := by
2   by_cases h : n = 0
3   · exact 0r.inl h
4   · exact 0r.inr h
5
6 theorem em_nat' (n : Nat) : n = 0 ∨ n ≠ 0 :=
7   if h : n = 0 then 0r.inl h else 0r.inr h

```

exfalso

Replaces any goal with **False**. Useful when the hypotheses are contradictory and the goal itself is irrelevant:

```

1 theorem from_absurd_bounds (n : Nat) (h1 : n < 2) (h2 : 2 < n) :
2   n = 42 := by
3   exfalso
4   omega

```

subst

Given **hab** : $a = b$ where one side is a local variable, **subst hab** eliminates the variable, rewriting it away everywhere:

```

1 theorem subst_demo (a b : Nat) (hab : a = b) (ha : a ≤ 5) : b ≤ 5 := by
2   subst hab
3   exact ha

```

calc — and the Trans instances underneath

calc writes a chain of (in)equalities the way you would on paper. Each step is glued to the previous one by **Trans.trans**, a typeclass method — which is why Chapter 1's chain had to be stated with $\leq$ (core Lean ships a **Trans** instance for $\leq$ but not for $\geq$):

```

1 theorem calc_demo (a b c d : Nat)
2   (h1 : a ≤ b) (h2 : b ≤ c) (h3 : c ≤ d) : a ≤ d :=
3   calc a ≤ b := h1
4     _ ≤ c := h2
5     _ ≤ d := h3

```

```
6
7 theorem calc_demo' (a b c d : Nat)
8   (h1 : a ≤ b) (h2 : b ≤ c) (h3 : c ≤ d) : a ≤ d :=
9   Trans.trans (Trans.trans h1 h2) h3
```

split

Where `cases` splits on a *value*, `split` splits on the control flow of the *goal*: an `if` (or `match`) in the target becomes one goal per branch, with the branch condition as a hypothesis.

```
1 def absDiff (a b : Int) : Int := if a ≤ b then b - a else a - b
2
3 theorem absDiff_nonneg (a b : Int) : 0 ≤ absDiff a b := by
4   simp only [absDiff]
5   split <=> omega
```

The combinator `t1 <=> t2` runs `t1`, then runs `t2` on *every* goal it produced — the workhorse of this book’s “case-split everything, then `simp/omega`” proofs.

change

Replaces the goal by a *definitionally equal* one — nothing is proved, the goal is merely re-displayed in a shape other tactics can match:

```
1 example : (fun n : Nat => n + 2) 3 = 5 := by
2   change 3 + 2 = 5
3   rfl
```

simp and simp_all

`simp` rewrites the goal to normal form with a database of equations (extendable per call: `simp [FlatNat.le]` also unfolds that definition). `simp_all` does the same to the hypotheses *and* uses them as rewrite rules — the closer for most of this book’s case-bash proofs.

```
1 theorem simp_demo (xs : List Nat) : (xs ++ []).length = xs.length := by
2   simp
3
4 theorem simp_all_demo (xs : List Nat) (h : xs = [1, 2]) :
5   xs.length = 2 := by
6   simp_all
```

`simp only [f]` restricts to the given lemmas — predictable, no database.

omega and decide

omega is a decision procedure for linear arithmetic over **Int** and **Nat** (with $+$, $-$, $\leq$, $<$, $=$, $\min/\max$, multiplication by constants). **decide** evaluates any *decidable* proposition to **true** — it works whenever there is a finite computation to run, e.g. statements about all **Bools**.

```

1 example (a b : Nat) (h : a + 2 * b = 10) (hb : 3 ≤ b) : a ≤ 4 := by
2   omega
3
4 example : (15 % 4 = 3) ∧ (2 ^ 10 = 1024) := by decide
5 example : ∀ b : Bool, (b && !b) = false := by decide

```

have and show

have $h : P := \text{proof}$ adds an intermediate fact; **show** P restates the current goal (up to definitional equality — a gentler **change**, often used purely for readability):

```

1 theorem have_demo (a : Nat) (h : a ≤ 3) : a ≤ 10 := by
2   have h' : (3 : Nat) ≤ 10 := by omega
3   show a ≤ 10
4   exact Nat.le_trans h h'

```

A **have** with no proof term, like Chapter 7's **have** $\text{hab}' : \text{loLe } l1 \ l2 \wedge \text{hiLe } h2 \ h1 := \text{hab}$, re-types an existing hypothesis at a definitionally equal type — no work, just unfolding.

funext and congr

funext reduces $f = g$ to $\forall x, f\ x = g\ x$ (functions are equal when they agree everywhere). **congr** 1 goes the other way for constructors: it reduces $\langle a_1, b_1 \rangle = \langle a_2, b_2 \rangle$ to $a_1 = a_2$ and $b_1 = b_2$, closing proof fields automatically by proof irrelevance:

```

1 theorem funext_demo : (fun n : Nat => n + 0) = (fun n : Nat => n) := by
2   funext n
3   rfl
4
5 structure Point where
6   x : Int
7   y : Int
8
9 theorem congr_demo (p q : Point) (hx : p.x = q.x) (hy : p.y = q.y) :
10   p = q := by
11   cases p; cases q
12   congr 1 -- splits ⟨x₁,y₁⟩ = ⟨x₂,y₂⟩ into x₁ = x₂ and y₁ = y₂

```

(Here `congr 1` leaves the two component goals, which `hx` and `hy` close — after `cases`, they already *are* those goals, so Lean finds them by assumption.)

A.4 Programming Constructs

do-notation, mut, for, while, and Id.run

A `do` block sequences actions in a monad. Inside it, `let x ← a` runs the action `a` and binds its result; `let mut` declares a variable that later statements may reassign; `for` and `while` loop. None of this is primitive — the compiler *desugars* it: statements become nested calls of the monad’s bind operator, and every `mut` variable becomes an accumulator threaded through a fold. Purely functional code wearing an imperative costume:

```
1 def sumTo (n : Nat) : Nat := Id.run do
2   let mut acc := 0
3   for i in [0:n+1] do
4     acc := acc + i
5   return acc
6
7 #eval sumTo 10 -- 55
8
9 -- what the mut/for pair compiles down to (morally): a fold
10 def sumTo' (n : Nat) : Nat :=
11   (List.range (n + 1)).foldl (fun acc i => acc + i) 0
12
13 #eval sumTo' 10 -- 55
```

`Id` is the do-nothing monad ($\text{Id } \alpha = \alpha$), and `Id.run` just unwraps it, so `sumTo` is an ordinary pure function — this is how the Sudoku solver’s `Id.run do` blocks work. `while` loops need a monad that permits possible non-termination, such as `IO`:

```
1 def firstPowerAbove (bound : Nat) : IO Nat := do
2   let mut p := 1
3   while p ≤ bound do
4     p := p * 2
5   return p
6
7 #eval firstPowerAbove 100 -- 128
```

IO.Ref

Genuine mutable state — the heart of a propagator `Cell` — lives in `IO.Ref α`, a mutable reference allocated with `IO.mkRef`:

```

1 def counterDemo : IO Nat := do
2   let r ← IO.mkRef 0      -- allocate a mutable reference holding 0
3   r.set 5                 -- overwrite the contents
4   r.modify (· + 1)        -- apply a function to the contents
5   r.get                   -- read it back
6
7 #eval counterDemo         -- 6

```

partial def

Lean normally insists every function provably terminates. **partial def** opts a definition out of that check: it can still be compiled and run, but Lean treats its body as opaque for proofs. We used it for `inferExpr` in Chapter 9, whose recursion structure is obvious to a human and tedious for a termination checker:

```

1 partial def collatzSteps (n : Nat) : Nat :=
2   if n ≤ 1 then 0
3   else if n % 2 == 0 then 1 + collatzSteps (n / 2)
4   else 1 + collatzSteps (3 * n + 1)
5
6 #eval collatzSteps 6      -- 8

```

Whether *this* function terminates for all inputs is a famous open problem — exactly the kind of definition **partial** exists for.

Structures, typeclasses, and deriving

A **structure** bundles named fields (possibly including *proof* fields, like `Interval.range`'s `ok`); `cases/rcases` take values apart and `{...}` builds them. A **class** is a structure whose instances Lean finds automatically for square-bracket parameters like `[PartialOrder α]`; **class B extends A** embeds one class in another (our `Lattice` **extends** `PartialOrder`). Finally, **deriving instance** `DecidableEq` for `FlatNat` asks Lean to generate an instance mechanically — available for simple inductives without proof fields; Chapter 7's `Interval` needed a hand-written one.

A.5 Tactics at a Glance

Every tactic used in this book, in one table. Sections above give worked examples of each.

Tactic	Usage
<code>rfl</code>	Prove $a = a$, up to computation
<code>exact e</code>	Close the goal with term e
<code>apply f</code>	Match goal against f 's conclusion; args become goals
<code>intro h</code>	Move premise / $\forall$ -variable into hypotheses
<code>have h : P := e</code>	Add an intermediate fact
<code>show P</code>	Restate the goal (definitional equality)
<code>cases x</code>	One goal per constructor of x (sugar for <code>T.rec</code>)
<code>rcases h with ⟨a, b⟩</code>	<code>cases</code> with nested destructuring patterns
<code>obtain ⟨a, b⟩ := h</code>	Same as <code>rcases</code> , different spelling
<code>rename_i x</code>	Name the last inaccessible hypothesis
<code>induction n</code>	<code>cases</code> + induction hypotheses (sugar for <code>T.rec</code>)
<code>constructor</code>	Apply the goal type's constructor (<code>And</code> , <code>Iff</code> , ...)
<code>by_cases h : p</code>	Split on decidable p (sugar for dependent <code>if</code>)
<code>exfalso</code>	Replace the goal with <code>False</code>
<code>subst h</code>	Eliminate a variable using $h : x = e$
<code>calc</code>	Chain of steps glued by <code>Trans.trans</code>
<code>rw [h]</code>	Rewrite the goal with equation h
<code>simp</code>	Normalise with the simp-lemma database
<code>simp only [f]</code>	Simp restricted to the given lemmas
<code>simp_all</code>	Simp goal and hypotheses, using hypotheses
<code>split</code>	One goal per branch of an <code>if/match</code> in the goal
<code>change P</code>	Replace goal by definitionally equal P
<code>omega</code>	Decide linear arithmetic over <code>Int/Nat</code>
<code>decide</code>	Evaluate a decidable proposition
<code>funext</code>	$f = g$ from $\forall x, f\ x = g\ x$
<code>congr n</code>	Split constructor equality into field equalities
<code>t1 <;> t2</code>	Run $t2$ on every goal produced by $t1$
<code>•</code> (bullet)	Focus the first remaining goal

Table A.1: The complete tactic vocabulary of this book.

A.6 Unicode Input

Symbol	Input	Meaning
$\leq$	<code>\le</code>	Less-than-or-equal
$\geq$	<code>\ge</code>	Greater-than-or-equal
$\rightarrow$	<code>\to</code>	Function/implication arrow
$\leftarrow$	<code>\l</code>	Reverse arrow
$\leftrightarrow$	<code>\iff</code>	If and only if
$\forall$	<code>\fa</code>	For all
$\exists$	<code>\ex</code>	There exists
$\sqcap$	<code>\sqcap</code>	Meet (inf)
$\sqcup$	<code>\sqcup</code>	Join (sup)
$\perp$	<code>\bot</code>	Bottom
$\top$	<code>\top</code>	Top
$\in$	<code>\in</code>	Set membership
$\notin$	<code>\notin</code>	Non-membership
α	<code>\a</code>	Greek alpha
β	<code>\b</code>	Greek beta
$\mathbb{N}$	<code>\N</code>	Natural numbers
$\wedge$	<code>\and</code>	Logical and
$\vee$	<code>\or</code>	Logical or
$\neg$	<code>\n</code>	Logical not

Table A.2: Lean 4 Unicode inputs used in this book.

Selected Solutions

This appendix contains solutions to exercises with a single definitive Lean 4 answer. Open-ended exercises are left to the reader. All solutions assume the book's definitions are in scope — `open` `OrderBook` for Part I, and additionally `namespace` `Propagators` for the Chapter 6 solutions.

Chapter 1

Exercise 1.1 (Reflexivity and symmetry)

```
1 theorem symm_antisymm_eq {α : Type} (r : Rel α)
2   (hs : Symmetric r) (ha : Antisymm r) :
3   ∀ a b : α, r a b → a = b := by
4   intro a b h
5   exact ha a b h (hs a b h)
```

Exercise 1.3 (Product order)

```
1 instance [OrderBook.PartialOrder α] [OrderBook.PartialOrder β] :
2   OrderBook.PartialOrder (α × β) where
3   le p q := PartialOrder.le p.1 q.1 ∧ PartialOrder.le p.2 q.2
4   le_refl := fun ⟨a, b⟩ =>
5     ⟨PartialOrder.le_refl a, PartialOrder.le_refl b⟩
6   le_antisymm := fun ⟨a1,b1⟩ ⟨a2,b2⟩ ⟨h1,h2⟩ ⟨h3,h4⟩ => by
7     have := PartialOrder.le_antisymm _ _ h1 h3
8     have := PartialOrder.le_antisymm _ _ h2 h4
9     simp_all
10  le_trans := fun ⟨a1,b1⟩ ⟨a2,b2⟩ ⟨a3,b3⟩ ⟨h1,h2⟩ ⟨h3,h4⟩ =>
11    ⟨PartialOrder.le_trans _ _ _ h1 h3,
12      PartialOrder.le_trans _ _ _ h2 h4⟩
```

Chapter 2

Exercise 2.1 (Monotone composition)

```
1 -- NB: no [LE _] binders – our PartialOrder already provides the ≤
2 -- notation; a second LE instance would introduce an unrelated order
```

```

3  -- and the hypotheses would no longer match Monotone's.
4  theorem antitone_comp_antitone {α β γ : Type}
5    [PartialOrder α] [PartialOrder β] [PartialOrder γ]
6    {f : α → β} {g : β → γ}
7    (hf : ∀ a b : α, a ≤ b → f b ≤ f a)
8    (hg : ∀ a b : β, a ≤ b → g b ≤ g a) :
9    Monotone (g ∘ f) where
10   map_le := fun a b h => hg (f b) (f a) (hf a b h)

```

Exercise 2.2 (Fixed points — ascending chain)

```

1  def iter {α : Type} (f : α → α) : Nat → α → α
2    | 0,      x => x
3    | n + 1, x => f (iter f n x)
4
5  theorem iter_chain {α : Type} [PartialOrder α]
6    {f : α → α} (hf : Monotone f)
7    {a : α} (ha : a ≤ f a) :
8    ∀ n : Nat, iter f n a ≤ iter f (n + 1) a := by
9    intro n
10   induction n with
11   | zero => exact ha
12   | succ n ih => exact hf.map_le _ _ ih

```

Exercise 2.3 (Identity is monotone)

```

1  theorem Monotone.id {α : Type} [PartialOrder α] :
2    Monotone (id : α → α) where
3    map_le := fun _ _ h => h

```

Chapter 3

Exercise 3.1 (Interval information order)

```

1  structure Interval where
2    l      : Int
3    h      : Int
4    l_le_h : l ≤ h
5
6  instance : PartialOrder Interval where
7    -- Information order: i ≤ j iff j is contained in i
8    -- (narrower = more informative = higher).
9    le := fun i j => i.l ≤ j.l ∧ j.h ≤ i.h
10   le_refl := fun a => ⟨Int.le_refl a.l, Int.le_refl a.h⟩
11   le_antisymm := fun a b hyp1 hyp2 => by

```



```

12   cases a; cases b
13   rename_i l1 h1 l1_le_h1 l2 h2 l2_le_h2
14   congr 1
15   · exact Int.le_antisymm hyp1.left hyp2.left
16   · exact Int.le_antisymm hyp2.right hyp1.right
17   -- the l_le_h field is closed by proof irrelevance
18   le_trans := fun a b c hyp1 hyp2 => by
19     rcases a with (a_l, a_h, a_le)
20     rcases b with (b_l, b_h, b_le)
21     rcases c with (c_l, c_h, c_le)
22     have hl_ab : a_l ≤ b_l := hyp1.left
23     have hl_bc : b_l ≤ c_l := hyp2.left
24     have hh_ba : b_h ≤ a_h := hyp1.right
25     have hh_cb : c_h ≤ b_h := hyp2.right
26     exact (Int.le_trans hl_ab hl_bc, Int.le_trans hh_cb hh_ba)

```

Meets always exist: the hull $[\min(l_1, l_2), \max(h_1, h_2)]$ is the greatest lower bound. Joins exist exactly when the intervals overlap — for disjoint intervals every candidate join would have to be an “empty interval”, which this type cannot represent. Chapter 7’s lattice adds it (as $\top$), plus unbounded ends (as $\perp$).

Exercise 3.3 (Characterising the order)

```

1  theorem le_iff_sup_eq {α : Type} [Lattice α] {a b : α} :
2    a ≤ b ↔ a ∪ b = b := by
3    apply Iff.intro
4    · intro h
5      apply PartialOrder.le_antisymm
6      · apply Lattice.le_sup
7        · exact h
8        · exact PartialOrder.le_refl b
9      · exact Lattice.sup_le_right a b
10   · intro h
11     rw [← h]
12     exact Lattice.sup_le_left a b

```

Exercise 3.4 (Product lattice)

```

1  instance {α β : Type} [Lattice α] [Lattice β] : Lattice (α × β) where
2    le      := fun a b => (a.fst ≤ b.fst) ∧ (a.snd ≤ b.snd)
3    le_refl := fun a => by
4      apply And.intro
5      · apply PartialOrder.le_refl
6      · apply PartialOrder.le_refl
7    le_antisymm {a b} := fun hyp1 hyp2 => by
8      rcases a with (l1, r1); rcases b with (l2, r2); congr 1

```

```

9   · exact PartialOrder.le_antisymm _ _ hyp1.left hyp2.left
10  · exact PartialOrder.le_antisymm _ _ hyp1.right hyp2.right
11  le_trans {a b c} := fun hyp1 hyp2 => by
12    apply And.intro
13    · exact PartialOrder.le_trans _ _ _ hyp1.left hyp2.left
14    · exact PartialOrder.le_trans _ _ _ hyp1.right hyp2.right
15  inf := fun a b =>
16    ⟨Lattice.inf a.fst b.fst, Lattice.inf a.snd b.snd⟩
17  sup := fun a b =>
18    ⟨Lattice.sup a.fst b.fst, Lattice.sup a.snd b.snd⟩
19  inf_le_left := fun a b => ⟨Lattice.inf_le_left a.fst b.fst,
20    Lattice.inf_le_left a.snd b.snd⟩
21  inf_le_right := fun a b => ⟨Lattice.inf_le_right a.fst b.fst,
22    Lattice.inf_le_right a.snd b.snd⟩
23  le_inf := fun a b c hyp1 hyp2 =>
24    ⟨Lattice.le_inf a.fst b.fst c.fst hyp1.left hyp2.left,
25    Lattice.le_inf a.snd b.snd c.snd hyp1.right hyp2.right⟩
26  sup_le_left := fun a b => ⟨Lattice.sup_le_left a.fst b.fst,
27    Lattice.sup_le_left a.snd b.snd⟩
28  sup_le_right := fun a b => ⟨Lattice.sup_le_right a.fst b.fst,
29    Lattice.sup_le_right a.snd b.snd⟩
30  le_sup := fun a b c hyp1 hyp2 =>
31    ⟨Lattice.le_sup a.fst b.fst c.fst hyp1.left hyp2.left,
32    Lattice.le_sup a.snd b.snd c.snd hyp1.right hyp2.right⟩

```

Chapter 6

Exercise 6.1 (Three-cell multiplication constraint)

The pattern mirrors the addition network: one forward propagator and two backward propagators, one for each variable that can be derived from the other two. Division by zero writes `.conflict`.

```

1  def mulProp (cx cy cz : Cell FlatNat) : PropStep := do
2    let x ← cx.read
3    let y ← cy.read
4    match x, y with
5    | .known a, .known b => cz.write (.known (a * b))
6    | _, _ => return false
7
8  -- z / x → y (or conflict if x does not divide z)
9  def divProp (cz cx cy : Cell FlatNat) : PropStep := do
10    let z ← cz.read
11    let x ← cx.read
12    match z, x with

```

```

13 | .known z, .known x =>
14 | if x | z then cy.write (.known (z / x))
15 |     else cy.write (.conflict)
16 | _, _ => return false
17
18 def mulNetwork : IO Unit := do
19   let cx ← Cell.new (α := FlatNat)
20   let cy ← Cell.new (α := FlatNat)
21   let cz ← Cell.new (α := FlatNat)
22
23   let _ ← cx.write (.known 3)
24   let _ ← cz.write (.known 12)
25
26   let net : Network := Network.mk #[
27     mulProp cx cy cz,
28     divProp cz cx cy,    -- z / x → y
29     divProp cz cy cx    -- z / y → x
30   ]
31   net.run
32
33   let yVal ← cy.read
34   match yVal with
35   | .known n  => IO.println s!"y = {n}"    -- y = 4
36   | .unknown => IO.println "y is unknown"
37   | .conflict => IO.println "contradiction"
38
39 #eval mulNetwork    -- y = 4

```

The three propagators cover all three “solve for one variable” cases. Remove `divProp cz cx cy` and `y` stays unknown; remove `mulProp` and you lose the ability to derive `cz` from both inputs.

Exercise 6.2 (Conflict detection)

Two independent writes of different values to the same cell produce `.conflict` via the join in `Cell.write`: `known 3` $\sqcup$ `known 5` = `conflict`. No propagators are needed — the conflict arises purely from the two seeds.

```

1 def conflictExample : IO Unit := do
2   let cx ← Cell.new (α := FlatNat)
3
4   -- Two independent sources disagree about x
5   let _ ← cx.write (.known 3)
6   let _ ← cx.write (.known 5)
7
8   let xVal ← cx.read
9   match xVal with

```

```
10 | .conflict => IO.println "contradiction (expected)"
11 | .known n  => IO.println s!"x = {n}"
12 | .unknown  => IO.println "x unknown"
13
14 #eval conflictExample    -- contradiction (expected)
```

The key is in `Cell.write`: it merges the new value with the old via `sup`, so the second write computes `known 3  $\sqcup$  known 5 = conflict` and stores that. From this point any further write leaves `conflict` unchanged (since `conflict  $\sqcup$  x = conflict` for all `x`), and any propagator reading `cx` will receive `conflict` and propagate it downstream.

To demonstrate downstream propagation, add a dependent cell:

```
1 def conflictPropagates : IO Unit := do
2   let cx  ← Cell.new (α := FlatNat)
3   let cy  ← Cell.new (α := FlatNat)
4   let csum ← Cell.new (α := FlatNat)
5
6   let _ ← cx.write (.known 3)
7   let _ ← cx.write (.known 5)    -- cx is now conflict
8   let _ ← cy.write (.known 7)
9
10  let net : Network := Network.mk #[addProp cx cy csum]
11  net.run
12
13  let s ← csum.read
14  match s with
15  | .conflict => IO.println "sum is conflict (expected)"
16  | .known n  => IO.println s!"sum = {n}"
17  | .unknown  => IO.println "sum unknown"
18
19 #eval conflictPropagates    -- sum is conflict (expected)
```

Further Reading

- **Davey & Priestley**, *Introduction to Lattices and Order*, 2nd ed., Cambridge University Press, 2002. The standard introductory text on lattice theory, accessible and rigorous.
- **Radul & Sussman**, *The Art of the Propagator*, MIT CSAIL Technical Report, 2009. The foundational paper for the propagator model used in this book.
- **Tate, Stepp, Lerner & Chambers**, *Equality Saturation: a New Approach to Optimization*, POPL 2009. An application of lattice-based fixed-point algorithms to compiler optimization.
- **Nielson, Nielson & Hankin**, *Principles of Program Analysis*, Springer, 1999. Covers dataflow analysis (a lattice-based framework) and abstract interpretation.
- **The Lean 4 Documentation**, <https://lean-lang.org>. Official reference for Lean 4 syntax, tactics, and the core library.
- **Mathlib4**, https://leanprover-community.github.io/mathlib4_docs/. The community mathematics library for Lean 4. Once you have finished this book, exploring `Mathlib.Order` will show you how the constructions here scale to a large, verified library.